

Programming in VC++

MODULE 4.4



Centre for Electronics Design & Technology of India

A Scientific Society under Department of Electronics,
Govt. of India, New Delhi

**Published by CFS Documentation cell
Division of Centre for Electronics Design and Technology of India
A Scientific Society of Department of Electronics,
Govt. of India,
New Delhi.**

First Edition: 1999

TRADE MARKS: All brand name and product names mentioned in this book are trade marks or registered trade mark of their respective companies.

Every effort has been made to supply complete and accurate information. However, CEDTI assumes no responsibility for its use, nor for any infringement of the intellectual property rights of third parties which would result from such use.

No part of this publication may be stored in a retrieval system, transmitted or reproduced in any forms or by any means, electronic, photocopy, photograph, magnetic or otherwise, without written permission of CEDTI.

CEDTI/CFS/99/4/4.4/R1

FOREWORD

The Information Technology and Telecom sectors have suddenly opened up avenues, which require a very large specially trained manpower. These sectors are highly dynamic and need training and re-training of manpower at a rapid rate. The growing gap of requirement of the industry and its fulfilment has created a challenging situation before manpower training institutes of the country. To meet this challenge most effectively, Centre for Electronics Design and Technology of India (CEDTI) has launched its nation-wide franchising scheme.

Centre for Electronics Design and Technology of India (CEDTI) is an Autonomous Scientific Society under the Govt. of India, Department of Electronics with its Headquarters at New Delhi. It operates seven centres located at Aurangabad, Calicut, Gorakhpur, Imphal, Mohali, Jammu and Tezpur. The scheme will be implemented and coordinated by these centres.

The scheme endeavours to promote high quality computer and information technology education in the country at an affordable cost while ensuring uniform standards in order to build a national resource of trained manpower. Low course fees will make this education available to people in relatively small, semi urban and rural areas. State-of-the-art training will be provided keeping in view the existing and emerging needs of the industrial and Govt. sectors. The examinations will be conducted by CEDTI and certificates will also be awarded by CEDTI. The scheme will be operated through all the seven centres of CEDTI.

The CEDTI functions under the overall control and guidance of the Governing Council with Secretary, Department of Electronics as its Chairman. The members of the council are drawn from scientific, government and industrial sectors. The Centres have separate executive committees headed by Director General, CEDTI. The members of these committees are from academic/professional institutes, state governments, industry and department of electronics.

CEDTI is a quality conscious organisation and has taken steps to formally get recognition of the quality and standards in various activities. CEDTI, Mohali was granted the prestigious ISO 9002 certificate in 1997. The other centres have taken steps to obtain the certification as early as possible. This quality consciousness will assist CEDTI in globalizing some of its activities. In keeping with its philosophy of 'Quality in every Activity', CEDTI will endeavour to impart state of the art – computer and IT training through its franchising scheme.

The thrust of the Software Courses is to train the students at various levels to carry out the Management Information System functions of a medium sized establishment, manufacture Software for domestic and export use, make multimedia presentations for management and effectively produce various manufacturing and architectural designs.

The thrust of the Hardware Courses at Technician and Telecommunication Equipment Maintenance Course levels is to train the students to diagnose the faults and carry out repairs at card level in computers, instruments, EPABX, Fax etc. and other office equipment. At Engineer and Network Engineer levels the thrust is to train them as System Engineers to install and supervise the Window NT, Netware and Unix Networking Systems and repair Microcontrollers / Microprocessor based electronic applications.

An Advisory Committee comprising eminent and expert personalities from the Information Technology field have been constituted to advise CEDTI on introduction of new courses and revising the syllabus of existing courses to meet the changing IT needs of the trade, industry and service sectors. The ultimate objective is to provide industry-specific quality education in modular form to supplement the formal education.

The study material has been prepared by the CEDTI, document centre. It is based on the vast and rich instructional experience of all the CEDTI centres. Any suggestions on the improvement of the study material will be most welcome.

(R. S. Khandpur)

Director General (CEDTI)

Programming in Visual C++

Table of Contents

Chapter 1. An Introduction to MS Visual C++1

1.1 The Developer Studio User Interface

1.1.1 Creating the Application

1.1.2 Developing the Application

1.2 How to Use AppWizard

1.2.1 Building and Running Application

1.3 How to use ClassWizard

1.4 Output and Error Messages

1.5 Creating DLLs, Console Applications and More...

1.6 Creating the AppWizard Starter Files

Chapter 2. Microsoft Foundation Classes10

2.1 MFC Classes

2.1.1 Application Class

2.1.2 Windows Classes

2.2 The Structure of an MFC Application

2.3 Examples of MFC

2.4 MFC Class Hierarchy

Chapter 3. The Document-View Architecture15

3.1 Document-View Application

3.1.1 Understanding the Document Class

3.1.2 Understanding the View Class

3.2 Exercise

Chapter 4. Messages and Message Handling21

4.1 Message Handling in MFC

4.2 Exercise

Chapter 5. Menus & Keyboard Accelerators24

5.1 Standard Menu and Commands

5.1.1 To create a menu on menu bar

5.1.2 To create a menu-item

5.2 Keyboard Accelerators

5.2.1 Adding an Accelerator key with menu-item

5.3 Exercise

Chapter 6. Graphics & Text Drawing27

6.1 The Graphical Device Independence(GDI)

6.2 The Device Context(DC)

6.3 GDI Objects

6.4 The CDC Class

6.5 The Device Context Classes

6.6 Creating a Pen

6.7 Graphical Objects

6.8 Exercise

Chapter 7. Windows Control31

7.1 Button Control

7.1.1 Button Styles

7.1.2 Button Messages

7.1.3 Creating a Button

7.2 Edit Control

7.2.1 Edit Control Styles

7.2.2 Edit Messages

7.2.3 CEdit Class Methods

7.2.4 Creating an Edit Control

7.3 List Box Control

7.3.1 List Box Control Styles

7.3.2 List Box Messages

7.3.3 CListBox Class Methods

7.3.4 Creating a List Box Control

7.4 Combo Box Control

7.3.1 Combo Box Control Styles

7.3.2 Combo Box Messages

7.3.3 CComboBox Class Methods

7.3.4 Creating a Combo Box Control

7.5 Exercise

Chapter 8. Dialog Boxes44

8.1 Introduction to Dialog boxes

8.1.1 Modal Dialog Box

8.1.2 Modeless Dialog Box

8.1.3 Dialog Templates

8.2 Constructing Dialogs

8.3 Invoking Dialogs

8.3.1 Modal dialog box

8.3.2 Modeless dialog box

8.3.3 Dialog Data Exchange

8.3.4 Dialog Data Validation

8.4 Common Dialogs

8.4.1 CColorDialog

8.4.2 CFileDialog

8.4.3 CFindReplaceDialog

8.4.4 CFontDialog

8.4.5 CPageSetupDialog

8.4.6 CPrintDialog

8.5 Exercise

Chapter 9. File Access50

9.1 Introduction to CFile Class

9.2 The CStdioFile Class

9.3 The CMemFile Class

9.4 Exercise

Chapter 10. Dynamic Link Libraries54

10.1 The DLLs entry point

10.2 DLLs written with MFC

10.2.1 Regular DLL

10.2.2 Extension DLL

10.3 How to create a DLL

10.4 Exercise

Chapter 11. Database Programming58

11.1 The DLLs entry point

11.2 DLLs written with MFC

11.2.1 Regular DLL

11.2.2 Extension DLL

11.3 How to create a DLL

11.4 Exercise

Chapter 11. Database Programming58

11.1 ODBC Overview

11.2 MFC Database Classes

11.3 Creating Database Application

11.4 DAO Basics

1. AN INTRODUCTION TO MICROSOFT VISUAL C++

A Windows application is very complex, it is a veritable workhorse that handles much of your application's activity automatically. Windows applications usually have a main window with a menu bar, scroll bars, sizing buttons, and other controls all of which are handled to a great extent by Windows.

A Windows program can receive hundreds of different messages while it's running. Your application determines whether to respond to these messages or to ignore them. If the user clicks the mouse pointer in your window, for example, your program gets a message for left mouse button down(WM_LBUTTONDOWN). The program can handle the message, performing the function the user requested.

Windows programming included a large number of Application Programming Interface(API). Windows, although not especially difficult, takes a lot of time and practice. Before you program your first application, you should be familiar with at least the most used functions in the API so that you know the tools you have at your disposal.

A Visual C++ application is an application for Windows that you design and developing using Microsoft Foundation Classes(MFC). The Microsoft Visual C++ build tools, and the Visual C++ Windows-based development environment. However, Microsoft's MFC goes a long way toward simplifying the process of writing Windows applications by hiding many of the details inside custom window classes. By using MFC, you can create a fully operational window with very few lines of code. By using a totally integrated environment, you can develop your applications by focusing on the Visual interface elements. Visual C++ calls these tools “*User Interface Objects*”. You first design the user-interface objects and then use Visual C++ tools to create and manage the code to support them. These tools automate the often tedious and error-prone process of deriving classes, creating member functions, and mapping functions to messages.

1.1 THE DEVELOPER STUDIO USER INTERFACE

Microsoft Visual C++ is one component of the Microsoft Developer Studio, shown in figure 1.1, is an *Integrated development environment (IDE)* because within a single tool, you can perform the following:

- Generate starter applications without writing code.
- View a project several different ways.
- Edit source and include files.
- Build the visual interface (menus and dialog boxes) of your application.
- Compile and link.
- Debug an application while it runs.

With Visual C++, you work on a single application as a project. A *project* is a collection of files: source, headers, resources, settings, and configuration information. Developer Studio is designed to enable work on all aspects of a single project at once. You create a new application by creating a new project. When you want to work on your application, you open the project (a file with the extension .dsw) rather than opening each code file independently. The project workspace window available under developer studio, which makes it simple and fast to access any part of your application.

Developer Studio uses two different files to keep track of all the information about your project. The *project workspace file*, with a .dsw extension, contains the names of all the files in the project, what directories they are in,

compiler and linker options, and other information required by everyone who may work on the project. There is also a *project file*, with a .dsp extension, for each project within the workspace. The *workspace options file*, with a .opt extension, contains all your personal settings for Developer Studio—colors, fonts, toolbars, which files are open and how their MDI windows are sized and located, breakpoints from your most recent debugging session, and so on. If someone else is going to work on your project, you give that person a copy of the project workspace file and project file but not the project options file.

To open the project, open the project workspace file. The other files are opened automatically.

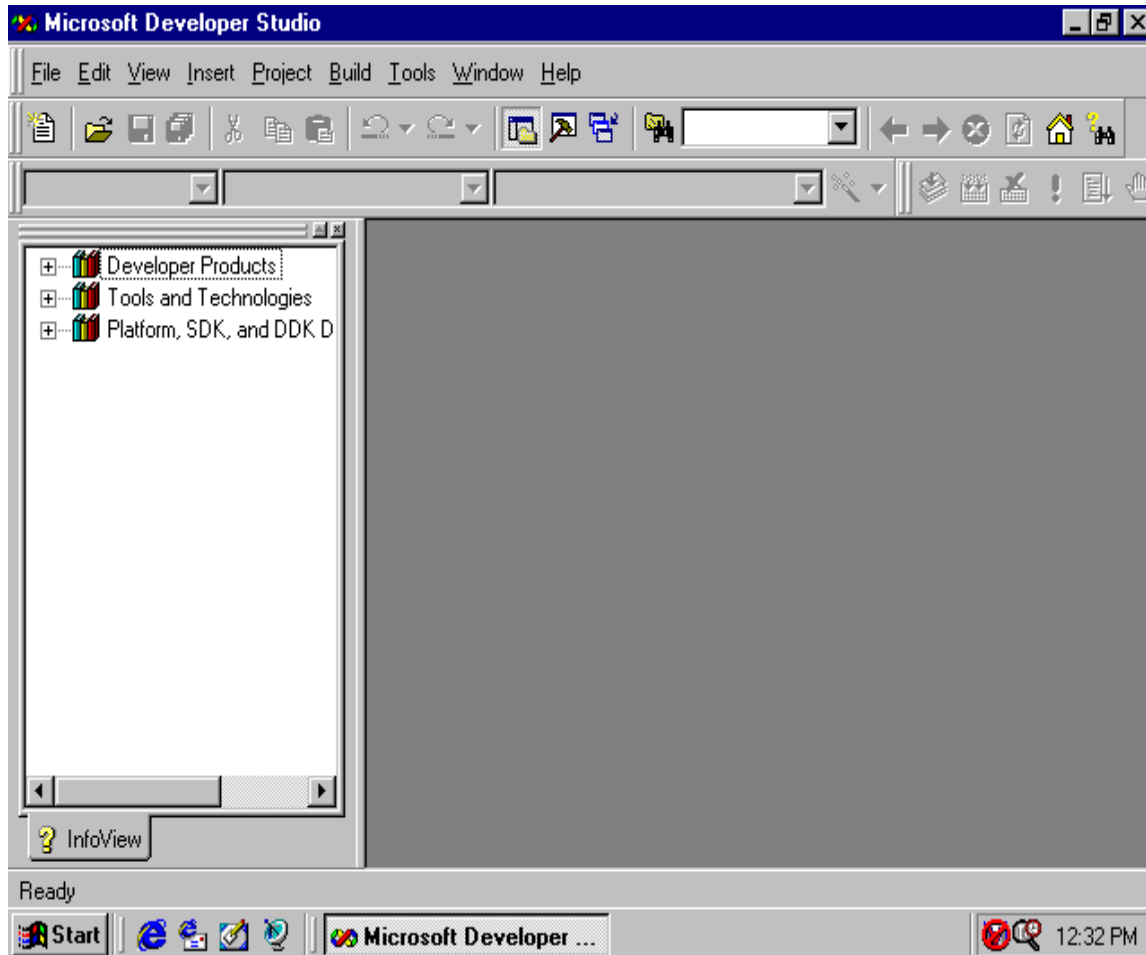


Figure 1.1 The Microsoft Developer Studio

There are four major components in Developer Studio. These are as follows :

- 1.1.1 The Info View
- 1.1.2 The Resource View
- 1.1.3 The Class View
- 1.1.4 The Files View

- **The Info view**
Find your way around Books Online(Help).
- **The Resource view**
Access your menus, dialog boxes, bitmaps, and other user interface resources.
- **The Class view**
See all your classes, and their variables and functions.
- **The File view**
Organize your files and open them easily.

When you first start Developer Studio and no project is open, the only tab in the Project Workspace window is the InfoView tab, as shown in the figure 1.1. When a project is open, there are other tabs to choose from.

Clicking the ResourceView tab in the Project Workspace window brings up an expandable and collapsible outline of the visual elements of your program: accelerators, dialog boxes, icons, menus, the string table, toolbars, and version information. These resources define the way users interact with your program.

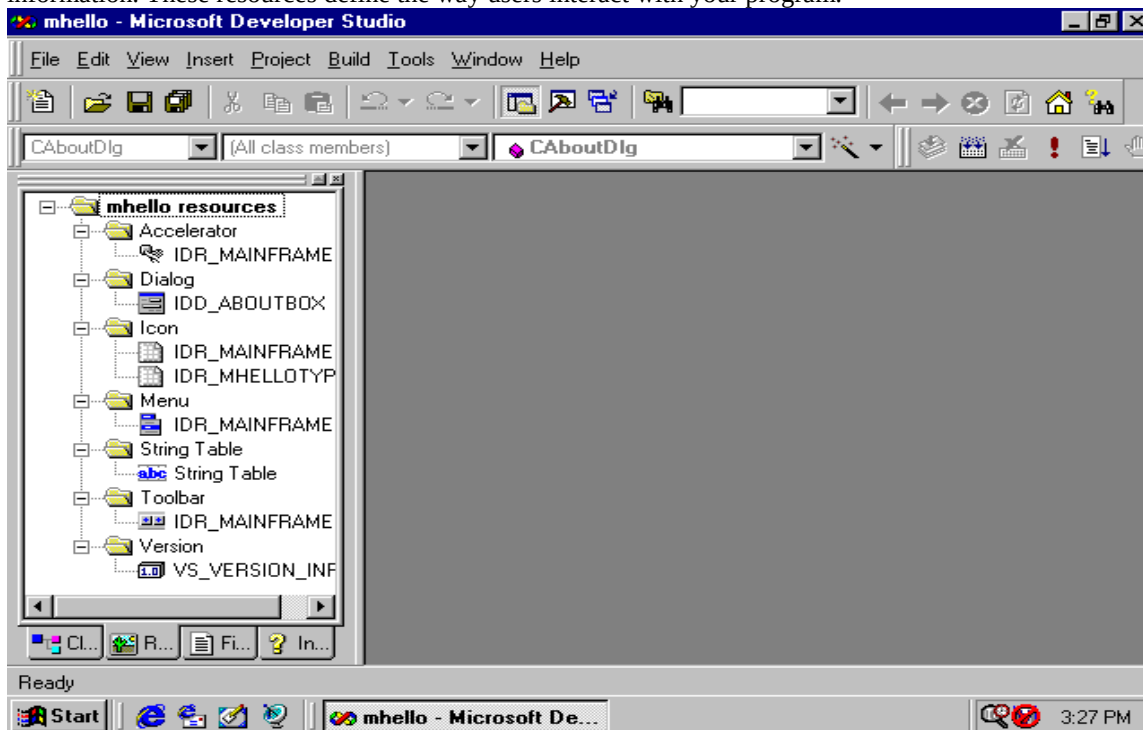


Figure 1.2 The ResourceView shows the files in your application.

The ClassView shows the classes in your application. Under each class, the member variables and functions are shown into your member functions, as demonstrated in Figure 1.3



Figure 1.3 The ClassView shows the functions and variables in each class in your application.

The FileView is much like the ClassView in that you can display and edit source and header files. However, it gives you access to parts of your file that are outside of class definitions, and makes it easy to open non-code files like resources and plain text.

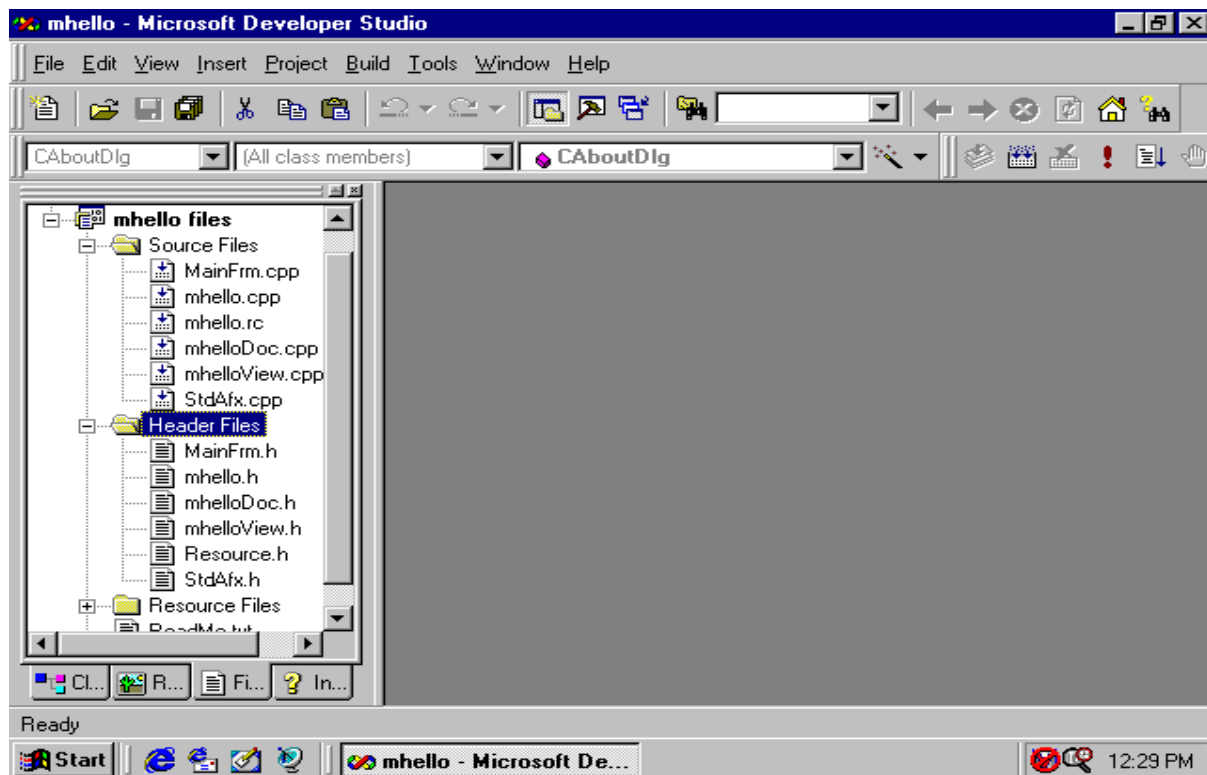


Figure 1.4 The FileView displays source and header files.

Developing a Visual C++ applications can be broken into two steps:

1.1.1 Creating the Applications

The first step in creating a Visual C++ application is to name a project, for example here is MHELLO.

After you name your project, use AppWizard to generate a set of application starter files. It is important to use AppWizard first during the development of a VC++ application, because it created source code that is a compatible with ClassWizard.

1.1.2 Developing the Application

The development stage for an application for windows involves editing source and resource files, compiling and linking, testing and debugging. These activities are repeated throughout a normal development cycle. There is an order involved, however, because you always create user-interface objects, first, then ClassWizard to create the code shell and the text editor to write the functional code.

1.2 HOW TO USE APPWIZARD ?

The first step in using AppWizard is to create a new project workspace. Begin by selecting the New from the File menu and select Project Workspace from the New dialog box. When the New Project Workspace dialog box appears, select MFC AppWizard(exe) as the type of the project.

You must also specify a name and a location for your project, such as \RSONI\MHELLO. Click the Create button to start AppWizard, shown in figure 1.5.

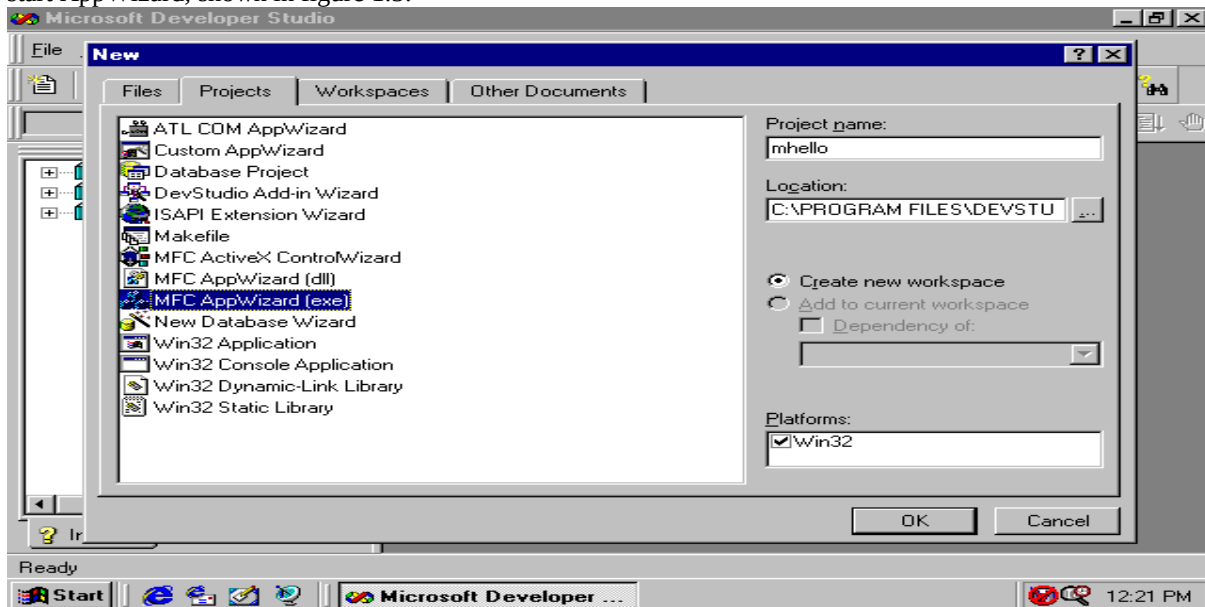


Figure 1.5 Create a New ProjectWorkspace

Application setups the following types of the generic programs for you:

- | | |
|-----------------------------------|---|
| Single Document Interface (SDI) | : A program that controls a single document at a time. |
| Multiple Document Interface (MDI) | : A program that controls a multiple documents at a time. |
| Dialog-Box based | : A program that has a dialog box as its main display window. |

The New Project dialog box appears, the dialog box requires that you enter the project name, project type and select a project location.

1. Choose SDI, MDI or Dialog-based application.
2. Choose database support, if any.
3. Choose OLE Control, if any.
4. Choose Finish.

1.2.1 Building And Running Application

1. Click the Build toolbar button or Build exec option from Build menu.
2. Choose Execute exe from the Build menu

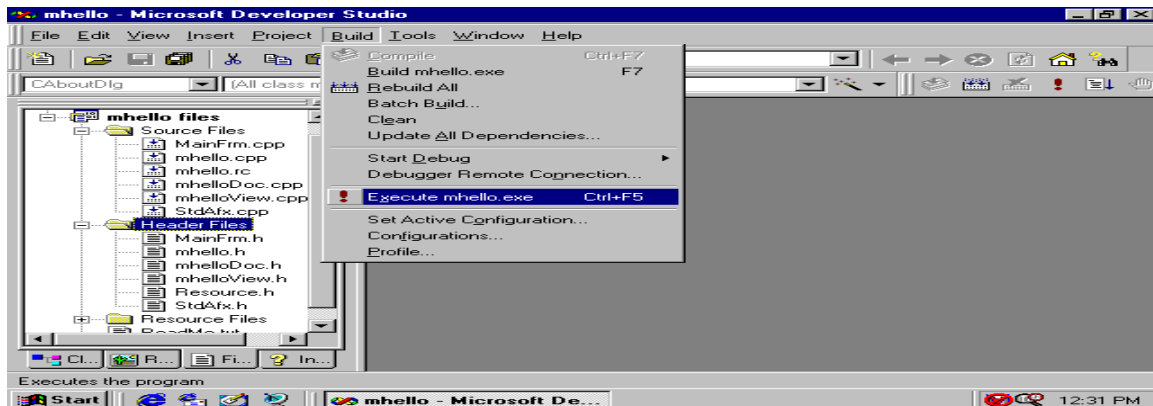


Figure 1.6 Build your Application

1.3 HOW TO USE CLASSWIZARD?

ClassWizard is like a programmer's assistant: it makes it easier for you to do certain routine tasks such as creating new classes, defining message handlers, overriding MFC virtual functions, and gathering data from controls in a dialog box, form view, or record view.

It is important to note that ClassWizard works only with applications that use MFC classes. Also, ClassView does not recognize classes unless they are registered in the ClassView database(CLV) file.

With ClassWizard, you can:

- Create new classes, derived from many of the main framework base classes that handle Windows messages and recordsets.
- Map messages to functions, associated with windows, dialog boxes, controls, menu items, and accelerators.
- Create new message-handling member functions.
- Delete message-handling member functions.
- Define member variables that automatically initialize, gather, and validate data entered into dialog boxes or form views.

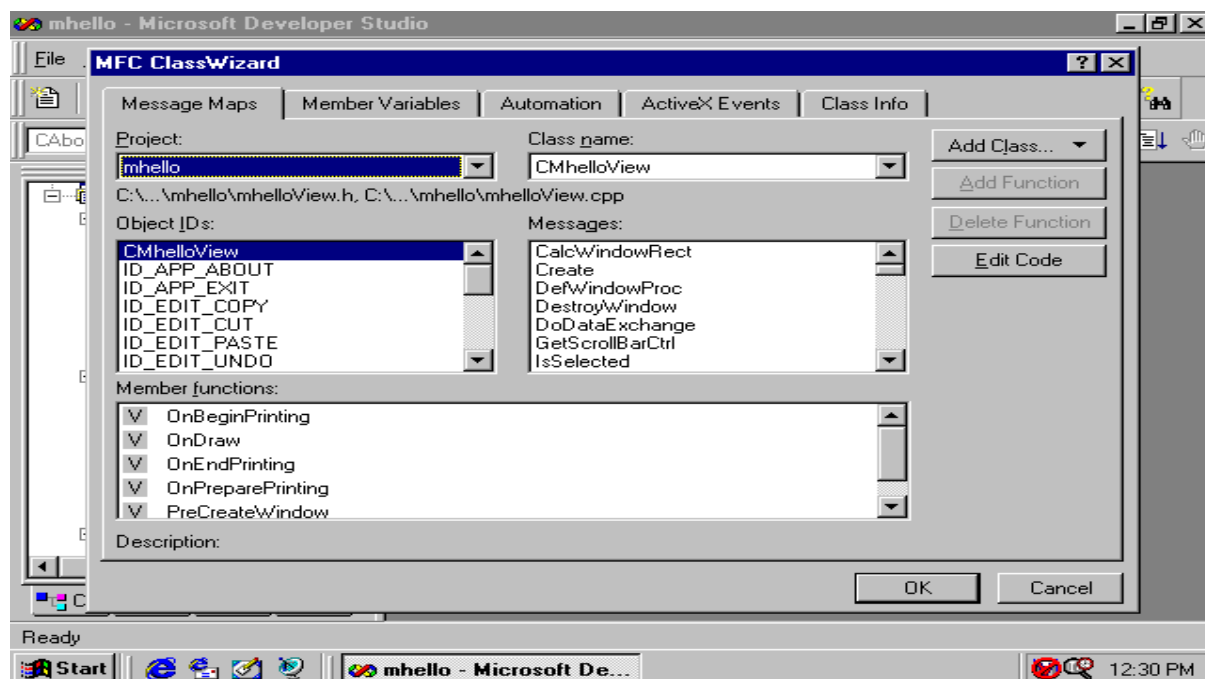


Figure 1.7 ClassWizard Window

1.4 OUTPUT AND ERROR MESSAGES

Across the bottom of the Developer Studio screen is the Output view. This is a tabbed view that shows output and error messages from a variety of Developer Studio functions.

If there is no Output view on your screen, choose View, Output from the menu to restore the view.

1.5 CREATING DLLS, CONSOLE APPLICATIONS, AND MORE...

Although most people use AppWizard to create an executable program, it can make many other kinds of projects. You choose **File, New** and then the **Projects** tab as discussed at the start of this chapter, but choose a different wizard from the list on the left of the **New** dialog box, shown in Figure 1.5. The following are some of the other projects AppWizard can create:

- ATL COM AppWizard
- Database Project
- DevStudio Add-in Wizard
- ISAPI Extension Wizard
- Makefile
- MFC ActiveX ControlWizard
- MFC AppWizard (dll)
- New Database Wizard
- Win32 Application
- Win32 Dynamic-Link Library

MFC AppWizard (DLL)

If you want to collect a number of functions into a DLL, and these functions use MFC classes, choose this wizard. (If the functions don't use MFC, choose **Win32 Dynamic-link Library**, discussed a little later in this section.)

Win32 Application

There are times when you want to create an application in Visual C++ that does not use MFC and does not start with the boilerplate code that AppWizard produces for you.

ATL COM AppWizard

ATL is the Active Template Library, and it's used to write small ActiveX controls.

Makefile

If you want to create a project that is used with a different "make" utility than Developer Studio, choose this wizard from the left-hand list in the **New Project Workspace** dialog box. No code is generated. If you don't know what a make utility is, don't worry—this wizard is for those who prefer to use a stand-alone tool to replace one portion of Developer Studio.

Database Project

If you have installed the Enterprise Edition of Visual C++, you can create a database project.

DevStudio Add-in Wizard

Add-ins are like macros that automate Developer Studio, but they are written in C++ or another programming language; macros are written in VBScript. They use Automation to manipulate Developer Studio.

ISAPI Extension Wizard

ISAPI stands for Internet Server API, and refers to functions you can call to interact with a running copy of Microsoft Internet Information Server, a World Wide Web server program that serves out Web pages in response to client requests.

MFC ActiveX Control Wizard

ActiveX controls are controls you write that can be used on a Visual C++ dialog, a Visual Basic form, even a Web page.

New Database Wizard

This wizard is for those with Visual InterDev installed. It simplifies connecting a Web page to an SQL database.

Win32 Dynamic-Link Library

If you plan to build a DLL that does not use MFC and does not need any boilerplate, choose the Win32 Dynamic-Link Library option rather than MFC AppWizard (dll). You get an empty project created right away with no questions.

1.6 CREATING AN APPWIZARD STARTER FILES

The AppWizard will generate a four-class starter application for your. These four classes are – the application class, the mainframe class, the document class, and the view class. The AppWizard setup allows you to make choice on the type of application that you want, e.g. SDI,MDI etc. Once these choices are made, AppWizard generates your starter code and create your project. The starter code is compatible with your ClassWizard.

Running AppWizard is a one-time task. Assuming you are making a typical application, you choose File, New, click the Projects tab, enter a name and folder, choose MFC AppWizard (exe), go through the six steps, create the application starter files and then never touch AppWizard again.

Here, the stepwise step documentation to generate your project files is described below:

1. Beginning from the Microsoft Developer Studio, choose File | New, and Select the Projects tab.
2. Choose Project Type from the given list, for example choose MFC AppWizard(.exe), and specify the location of the project folder.

When you have done this, choose Create and you will get the first step in a series of six query boxes, in which you will make choices about your application.

1. Step 1 gives the choices for your project interface(SDI, MDI, Dialog box). Select the document type (mainly SDI) and then Choose Next button to go to the next step.

An SDI application has menus that the user uses to open one document at a time and work with that document. This section presents the code that is generated when you create an SDI application with no database or compound document support, with a toolbar, a status bar, Help, 3-D controls, source file comments, and with the MFC library as a shared DLL. In other words, when you accept all the AppWizard defaults after Step 1.

2. Step 2 gives the query dialog box for the database supports.
3. Step 3 gives the query dialog box for the ActiveX, OLE Controls.
4. Step 4 gives the query dialog box for the printing and print preview, 3D Controls, Docking toolbar and the Help files.
5. Step 5 gives the dialog box for the linking of the DLLs into your application.
6. Step 6 gives the dialog box for the names of your classes. Choose Finish from this query dialog box.
7. Now you get a final display of “New Project Information” shown in Figure 1.8. Choose OK and your starter application source code file will be generated.

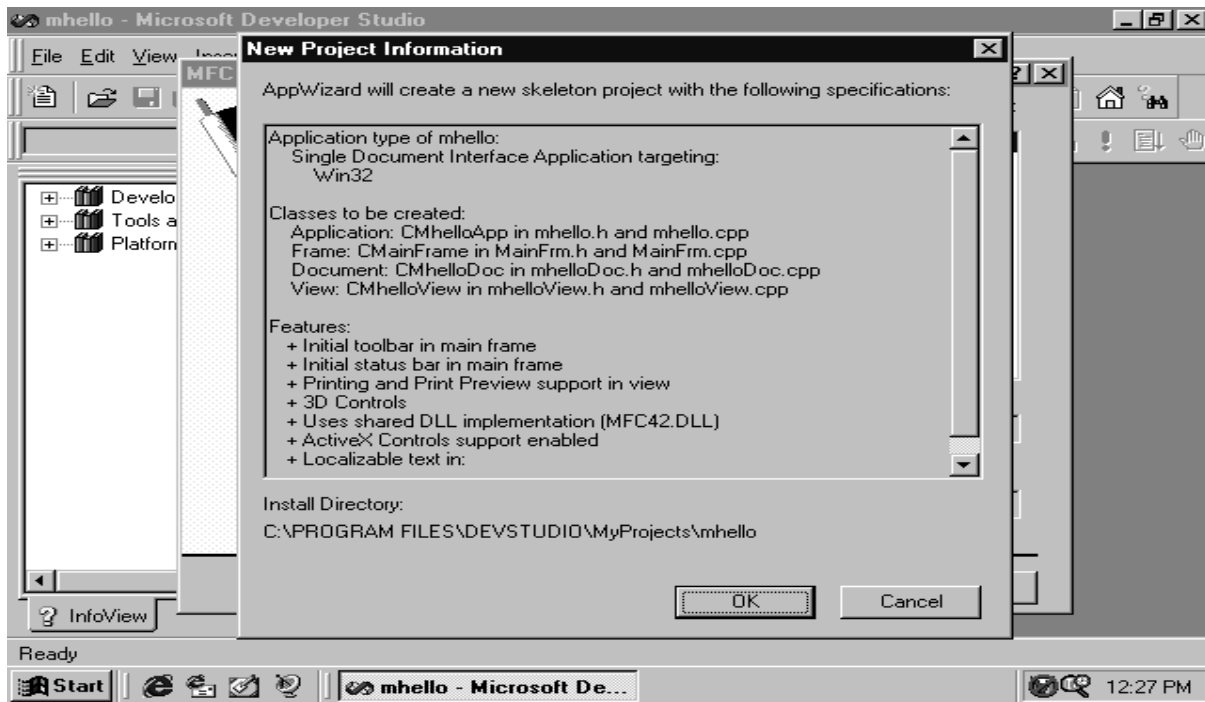


Figure 1.8 Display the AppWizard Starter Application Information

Five classes have been created for you. For the application called MHello, they are as follows:

- CAboutDlg, a dialog class for the About dialog box
- CMhelloworldApp, a CWinApp class for the entire application.
- CMhelloworldDoc, a Document class.
- CMhelloworldView, a View class.
- CMainFrame, a Frame class.

The details of each classes will be described later in this note.

2. MICROSOFT FOUNDATION CLASSES

The Microsoft Foundation Class Library (MFC) is an “*application framework*” for writing applications for Microsoft Windows and other platforms that support the Win32 API. The framework is implemented as a group of C++ classes, many of which represent common objects such as documents, Windows, dialog boxes, toolbars, and so on. The purpose of the framework is to reduce the effort required to design and implement application for Windows.

The application framework supplied by MFC is powerful and easy to reuse because the framework is an object-oriented class library. Instead of editing the framework source code directly, you derive new specified classes from those in the library. The derived classes inherit all of the behavior and functionality of their base classes, but you can extend them by overriding inherited member functions.

2.1 MFC CLASSES

MFC includes many classes you can use to write your applications more quickly and easily. These classes represent objects from which a Windows application is created objects such as windows, dialog boxes, menu bars, window controls, and many more, including some special objects such as status bars and control bars. In addition, MFC provides many general purpose classes for handling things like Strings, Arrays, Exception handling etc.

Specifically, MFC provides the following main categories of classes:

- Applications
- Windows
- Menus
- Dialog boxes
- Documents and Views
- Controls
- Graphics
- File and Database classes
- Various support classes
- Debugging and Exception classes
- Simple Data type classes

All the MFC classes are defined in the header file <afxwin.h>.

The CObject is the root class for all MFC classes; all of the MFC classes are derived from the CObject class. CObject provides basic services useful to all MFC objects, including their designed and created by you :-

- Serilization supports
- Runtime class information
- Object Diagnostic output
- Capability with collection classes

2.1.1 The Application Class

Every MFC program begins life as an application object. You derive this application object from MFC's CWinApp class, which provides initialization, runtime, and ending services for your Windows program. More specifically, CWinApp registers, creates, and shows an application's main window; sets the application's message loop running; and deletes the application when the user is finished. CWinApp also provides some additional services, such as providing access to the application's command line.

In the MFC hierarchy, CWinApp is derived from CWinThread, which is, in turn, derived from CCmdTarget. The CWinThread class takes care of thread-related tasks. Because every process running under Windows 95 requires at least one thread of execution, CWinApp is derived from CWinThread.

```
class CWinApp : public CWinThread
{
    DECLARE_DYNAMIC(CWinApp)
public:
    CWinApp(LPCTSTR lpszAppName = NULL); // Constructor
// Attributes
    HINSTANCE m_hInstance;
    HINSTANCE m_hPrevInstance;
    LPTSTR m_lpCmdLine;
    int m_nCmdShow;
    LPCTSTR m_pszAppName;
    LPCTSTR m_pszExeName;
    virtual BOOL InitApplication();
    virtual BOOL InitInstance();
    virtual int ExitInstance();
    virtual int Run();
    virtual ~CWinApp();
};
```

2.1.2 Window Classes

When you create a basic MFC application, it's up to you to tell MFC the type of main window it should create for the application. This is where MFC's window classes come in. MFC features several different types of windows. These windows include the following:

- Frame windows
- View windows
- Document windows
- Dialog boxes

All of the window classes are derived from the CWnd base class. The CWnd class provides the base functionality of all window classes in the Microsoft Foundation Class Library.

2.1.2.1 Frame Windows

Frame windows, represented by the CFrameWnd class, supply all the functionality of the CWnd class, and also handle document/view matters, the menu bar, control bars, accelerators. Frame windows also act as the base class from which four other MFC window classes are derived.

When, you'll derive the main window class for an SDI application from the CFrameWnd class. To derive for and MDI the window class is CMDIFrameWnd.

2.1.2.2 View Windows

The View windows handle the data associated with an application's window, MFC offers the document/view architecture. The CView class provides all the functionality needed by a generic view window. This includes displaying data in the frame window's client area and printing data.

MFC derives many specialized view classes from the general CView class. These include CCtrlView, CEditView, CListView, CRichEditView, CTreeView, CScrollView, CformView, CDaoRecordView, and CRecordView.

2.1.2.3 Document Windows

The `CDocument` class provides the basic functionality for user-defined document classes. A document separates the unit of data that the user typically opens with the File Open command and saves with the File Save command. The `CDocument` class supports standard operations such as creating a document, loading it, and saving it.

2.1.2.4 Dialog Boxes

Many Windows applications rely on dialog boxes to retrieve information from the user. MFC features a basic dialog box class, `CDialog`, which handles all the details of constructing, executing, and closing a dialog box.

2.2 THE STRUCTURE OF AN MFC APPLICATION

When writing a program, the first step is to define and create the application's main window, which is the "real estate" or piece of the screen that the application will control. A program's main window is also referred to as the mainframe.

In your MFC application, you derive your main window or mainframe class from the MFC class `CFrameWnd`, which is derived from `CWnd`. Every MFC application will have a mainframe class.

Another class that every MFC application must have is an application class. An application class is derived from the MFC class `CWinApp`.

My first Visual C++ program is to display a "Hello Visual C++ World!" in the caption bar of the mainframe window, shown in figure 2.1.

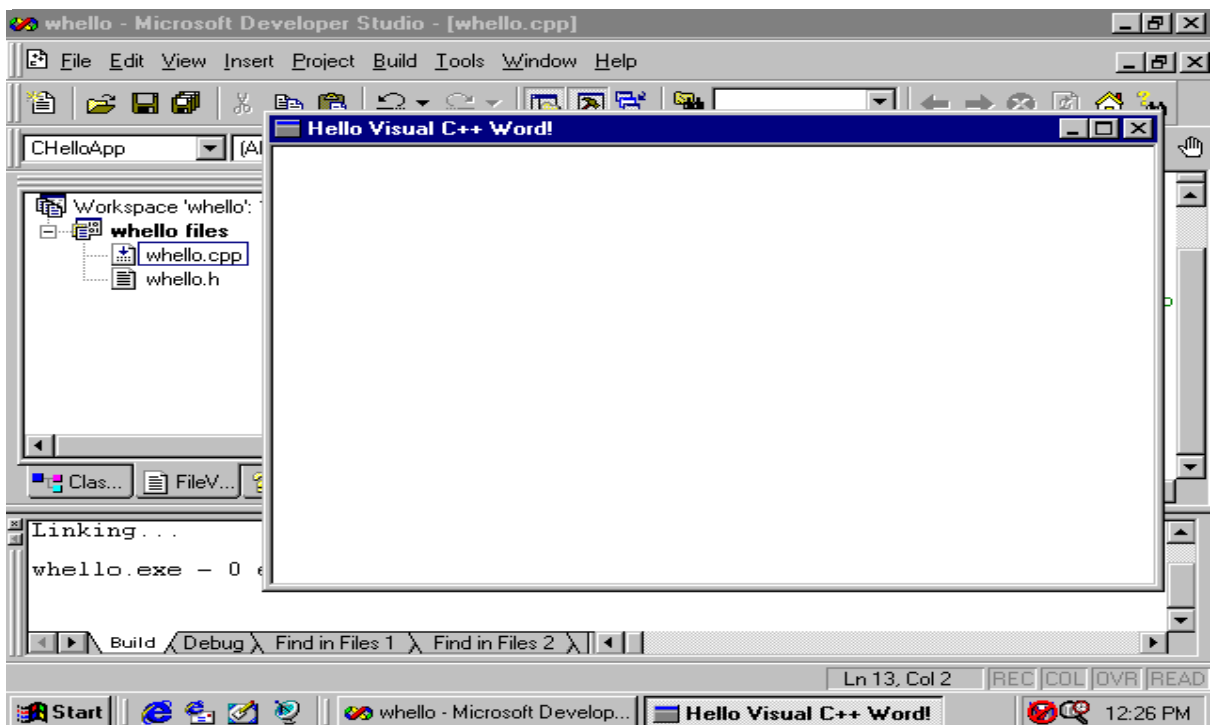


Figure 2.1 The Output Screen of Hello Visual C++ World Application

To create an minimal MFC classes based application, at least two classes `CFrameWnd` and `CWinApp` are required.

The code is separated into an application class, which has a header file (WHello.h) and an implementation file (WHello.cpp). The header file contains all the program variables and the structures for the functions. The listing of these files are given below:

// Header File whello.cpp

```
#include <afxwin.h>
class CHelloApp : public CWinApp {      // Application class
public :
    virtual BOOL InitInstance();
};

class CHelloWnd : public CFrameWnd { //Mainframe class or Frame Window
public :
    CHelloWnd() { Create(NULL,"Hello Visual C++ Word!");}
};
```

//Implementation file whello.cpp

```
#include "whello.h"
BOOL CHelloApp::InitInstance()
{
    CHelloWnd* pf= new CHelloWnd;
    pf->ShowWindow(SW_SHOWMAXIMIZED);
    pf->UpdateWindow();
    m_pMainWnd = pf;
    return TRUE;
}
CHelloApp happ;      // derive, create and run the application
```

2.3 EXAMPLES OF MFC

2.3.1 Controls

CButton	Create a Command button.
CEdit	Create an Edit Control.
CListBox	Create a ListBox control.
CComboBox	Create a ComboBox control.
CStatic	Create a Static control.
CRichEditCtrl	Create a Rich Edit Text Box control.

2.3.2 Dialog Boxes

CFileDialog	Create an Open and Save As File dialog box.
CColorDialog	Support for Windows color selection dialog box.
CFindReplaceDialog	Supports to use of the Find & Replace dialog box.
CFontDialog	Supports for font selection dialog box.
CPageSetupDialog	Supports to printer setup dialog box.
CPrintDialog	Supports to Print & Print Preview dialog box.

2.3.3 ODBC Database Support

CDatabase	Supports to database object under ODBC environment.
CRecordset	Supports to recordset object under ODBC environment.

2.3.4 DAO Database Support

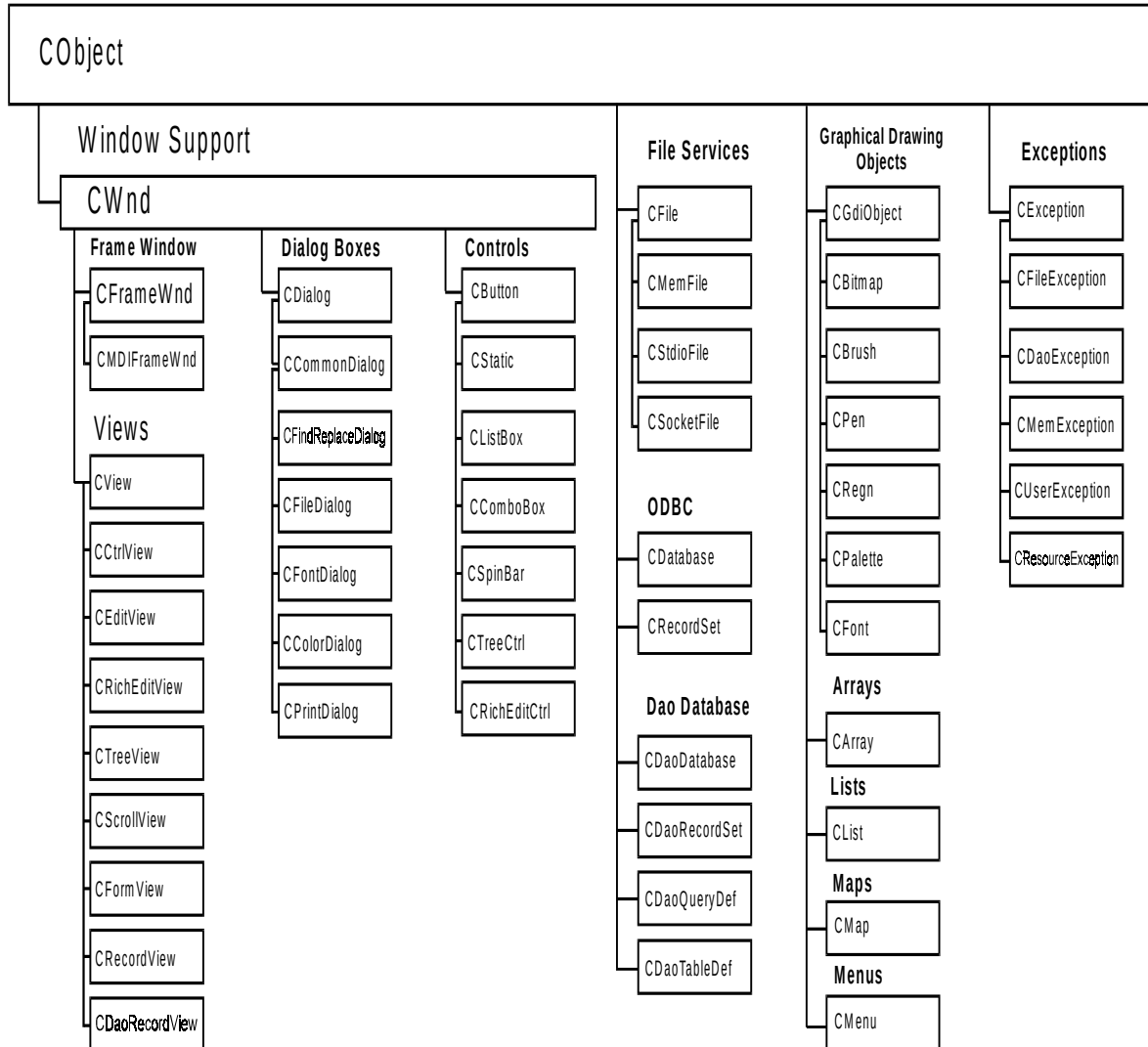
CDaoDatabase	Supports to database object under DAO environment.
CDaoRecordset	Supports to recordset object under DAO environment.

2.3.5 File Services

CFile
CStdioFile
CMemFile

Supports for a file handling supports under Windows 95/NT.
Supports to ASCII-character file handling.
Supports to memory files.

2.4 MFC CLASS HIERARCHY



3. THE DOCUMENT-VIEW ARCHITECTURE

Today MFC uses Document-View Architecture. The architecture separates an application data and the various views of data the application provides to the user.

The idea behind the Document-View architecture is to separate actual data from representation of that data. In other words, for one set of data, you can generate multiple views of it. A Spreadsheet is an example of it. A single spreadsheet can contains rows and columns of numbers. Yet from that data can generate a number of graphs. MFC provides several classes that implements the Document-View architecture. These includes the following :-

CDocument

The **CDocument** class provides the basic functionality for programmer-defined document classes. A document represents the unit of data that the user typically opens with the Open command on the File menu and saves with the Save command on the File menu.

CView

The **CView** class provides the basic functionality for programmer-defined view classes. A view is attached to a document and acts as an intermediary between the document and the user: the view renders an image of the document on the screen and interprets user input as operations upon the document. The view also renders the image for both printing and print preview.

The CView represents the actual windows you see on your screen. CView, which is derived from CWnd, is what you use to show your data to the outside world.

CDocTemplate/CSingleDocTemplate/CMultiDocTemplate

The document template is the glue that holds together a document and its view. It is the liaison between documents, views, and frameworks. A CSingleDocTemplate supports one document at a time; whereas a CMultiDocTemplate supports multiple documents simultaneously. Both are derived from the CDocTemplate class.

3.1 DOCUMENT-VIEW APPLICATION

The Document-View architecture uses the more complex MFC concepts of Document-View. The main advantage of the document-view application describes:

- The purposes of documents and views and how they interact in the framework.
- What you must do to implement them.

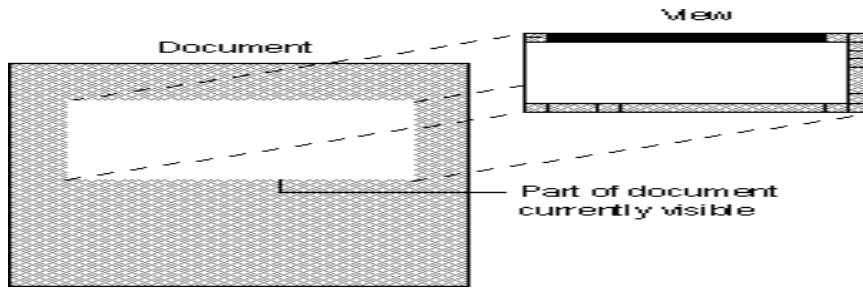


Figure 3.1 Document-View Application

The document-view implementation in the class library separates the data itself from its display and from user operations on the data. All changes to the data are managed through the document class. The view calls this interface to access and update the data.

Document and Views are the classes where most of your code will go when you write a Document-View application. A document contains the data for an application, which can be any unit of data archived to, and retrieved from a file. MFC provides the `CDocument` class that allow you to design your own document classes.

I have created an MFC based application using the document-view architecture (called as MHELLO project), which displays “Hello Visual C++ World!” into the windows caption. The listings and description of each files are explained as later.

3.1.1 Understanding the Document Class

When you looked over the various files generated by AppWizard, you discovered a class called `CMhelloDoc`, which was derived from MFC's `CDocument` class. In the MHELLO application, `CMhelloDoc` is the class from which the application instantiates its document object, which is responsible for holding the application's document data.

To see how this works, look at Listing 3.1, which shows the header file, AppWizard, created for the `CMhelloDoc` class.

Listing 3.1 MHELLODOC.H—The Header File for the `CMhelloDoc` Class

```
#if !defined(AFX_MHELLODOC_H__A465DFEB_CA50_11D2_BFD4_BCF297ECF366__INCLUDED_)
#define AFX_MHELLODOC_H__A465DFEB_CA50_11D2_BFD4_BCF297ECF366__INCLUDED_
#if _MSC_VER >= 1000
#pragma once
#endif // _MSC_VER >= 1000

class CMhelloDoc : public CDocument
{
protected: // create from serialization only
    CMhelloDoc();
    DECLARE_DYNCREATE(CMhelloDoc)
// Attributes
public:

// Operations
public:

// Overrides
    // ClassWizard generated virtual function overrides
    //{AFX_VIRTUAL(CMhelloDoc)
    public:
        virtual BOOL OnNewDocument();
```



```

        virtual void Serialize(CArchive& ar);
    //}}AFX_VIRTUAL

// Implementation
public:
    virtual ~CMHelloDoc();
#ifdef _DEBUG
    virtual void AssertValid() const;
    virtual void Dump(CDumpContext& dc) const;
#endif

// Generated message map functions
protected:
   //{{AFX_MSG(CMHelloDoc)
        // NOTE - the ClassWizard will add and remove member functions here.
        // DO NOT EDIT what you see in these blocks of generated code !
    //}}AFX_MSG
    DECLARE_MESSAGE_MAP()
};
////////////////////////////////////
//{{AFX_INSERT_LOCATION}}
// Microsoft Developer Studio will insert additional declarations immediately before the previous line.
#endif // !defined(AFX_MHELLODOC_H__A465DFEB_CA50_11D2_BFD4_BCF297ECF366__INCLUDED_)

```

Notice also in the class's header file that the CMHelloDoc class includes two virtual member functions called OnNewDocument() and Serialize(). MFC calls the OnNewDocument() function whenever the user chooses the File, New command. You can use this function to perform whatever initialization must be performed on your document's data. The Serialize() member function is where the document class loads and saves its data. When you build this chapter's sample program, you won't believe how easy it is to load and save data.

3.1.2 Understanding the View Class

The view class is responsible for displaying, and enabling the user to modify, the data stored in the document object. To do this, the view object must be able to obtain a pointer to the document object. After obtaining this pointer, the view object can access the document's data members in order to display or modify them. If you look at Listing 3.2, you can see how the CMHelloView class, obtains pointers to the document object.

Listing 3.2 MHELLOVIEW.H—The Header File for the CMHelloView Class

```

#if !defined(AFX_MHELLOVIEW_H__A465DFED_CA50_11D2_BFD4_BCF297ECF366__INCLUDED_)
#define AFX_MHELLOVIEW_H__A465DFED_CA50_11D2_BFD4_BCF297ECF366__INCLUDED_
#if _MSC_VER >= 1000
#pragma once
#endif // _MSC_VER >= 1000

class CMHelloView : public CView
{
protected: // create from serialization only
    CMHelloView();
    DECLARE_DYNCREATE(CMHelloView)

// Attributes
public:
    CMHelloDoc* GetDocument();

// Operations
public:

// Overrides

```

```

        // ClassWizard generated virtual function overrides
        //{AFX_VIRTUAL(CMhelloView)
        public:
        virtual void OnDraw(CDC* pDC); // overridden to draw this view
        virtual BOOL PreCreateWindow(CREATESTRUCT& cs);
        protected:
        virtual BOOL OnPreparePrinting(CPrintInfo* pInfo);
        virtual void OnBeginPrinting(CDC* pDC, CPrintInfo* pInfo);
        virtual void OnEndPrinting(CDC* pDC, CPrintInfo* pInfo);
        //}AFX_VIRTUAL

// Implementation
public:
    virtual ~CMhelloView();
#ifdef _DEBUG
    virtual void AssertValid() const;
    virtual void Dump(CDumpContext& dc) const;
#endif
// Generated message map functions
protected:
    //{AFX_MSG(CMhelloView)
        // NOTE - the ClassWizard will add and remove member functions here.
        // DO NOT EDIT what you see in these blocks of generated code !
    //}AFX_MSG
    DECLARE_MESSAGE_MAP()
};
#ifdef _DEBUG // debug version in mhelloView.cpp
    inline CMhelloDoc* CMhelloView::GetDocument() { return (CMhelloDoc*)m_pDocument; }
#endif
////////////////////////////////////
//{{AFX_INSERT_LOCATION}}
// Microsoft Developer Studio will insert additional declarations immediately before the previous line.
#endif // !defined(AFX_MHELLOVIEW_H__A465DFED_CA50_11D2_BFD4_BCF297ECF366__INCLUDED_)

```

In the GetDocument() function as returning a pointer to a CMhelloDoc object. Anywhere in the view class that you need to access the document's data, you can call GetDocument() to obtain a pointer to the document.

The listings 3.3 and 3.4 gives the outline structure for the MHELLO.H and MAINFRAME.H files.

Listing 3.3 MHELLO.H—The Header File for the CMhelloApp Class

```

#ifndef AFX_MHELLO_H__A465DFE5_CA50_11D2_BFD4_BCF297ECF366__INCLUDED_
#define AFX_MHELLO_H__A465DFE5_CA50_11D2_BFD4_BCF297ECF366__INCLUDED_
#if _MSC_VER >= 1000
#pragma once
#endif // _MSC_VER >= 1000
#ifdef _AFXWIN
    #error include 'stdafx.h' before including this file for PCH
#endif
#include "resource.h"    // main symbols

class CMhelloApp : public CWinApp
{
public:
    CMhelloApp();
// Overrides
    // ClassWizard generated virtual function overrides
    //{AFX_VIRTUAL(CMhelloApp)

```

```

    public:
        virtual BOOL InitInstance();
    //}}AFX_VIRTUAL
// Implementation
    //{{AFX_MSG(CMhelloApp)
    afx_msg void OnAppAbout();
        // NOTE - the ClassWizard will add and remove member functions here.
        // DO NOT EDIT what you see in these blocks of generated code !
    //}}AFX_MSG
    DECLARE_MESSAGE_MAP()
};
//{{AFX_INSERT_LOCATION}}
// Microsoft Developer Studio will insert additional declarations immediately before the previous line.
#endif // !defined(AFX_MHELLO_H__A465DFE5_CA50_11D2_BFD4_BCF297ECF366__INCLUDED_)

```

Listing 3.4 MAINFRAME.H—The Header File for the CMainFrame Class

```

#ifndef AFX_MAINFRM_H__A465DFE9_CA50_11D2_BFD4_BCF297ECF366__INCLUDED_
#define AFX_MAINFRM_H__A465DFE9_CA50_11D2_BFD4_BCF297ECF366__INCLUDED_
#if _MSC_VER >= 1000
#pragma once
#endif // _MSC_VER >= 1000

class CMainFrame : public CFrameWnd
{
protected: // create from serialization only
    CMainFrame();
    DECLARE_DYNCREATE(CMainFrame)
// Attributes
public:
// Operations
public:
// Overrides
    // ClassWizard generated virtual function overrides
    //{{AFX_VIRTUAL(CMainFrame)
    virtual BOOL PreCreateWindow(CREATESTRUCT& cs);
    //}}AFX_VIRTUAL
// Implementation
public:
    virtual ~CMainFrame();
#ifdef _DEBUG
    virtual void AssertValid() const;
    virtual void Dump(CDumpContext& dc) const;
#endif
protected: // control bar embedded members
    CStatusBar m_wndStatusBar;
    CToolBar m_wndToolBar;
// Generated message map functions
protected:
    //{{AFX_MSG(CMainFrame)
    afx_msg int OnCreate(LPCREATESTRUCT lpCreateStruct);
        // NOTE - the ClassWizard will add and remove member functions here.
        // DO NOT EDIT what you see in these blocks of generated code!
    //}}AFX_MSG
    DECLARE_MESSAGE_MAP()
};
//{{AFX_INSERT_LOCATION}}

```

```
// Microsoft Developer Studio will insert additional declarations immediately before the previous line.  
#endif // !defined(AFX_MAINFRM_H__A465DFE9_CA50_11D2_BFD4_BCF297ECF366__INCLUDED_)
```

3.2 EXERCISE

1. Implement the explained problem using document-view.

4. MESSAGES AND MESSAGE HANDLING

The Microsoft Foundation Class Library uses a mechanism called the “message map” to route Windows messages and commands to the windows, documents, views, and other objects in an MFC application. Message maps route Windows messages; commands from menus, toolbar buttons, and accelerators; and control-notification messages.

Message maps connect messages to message handler functions, or “handlers,” in the target object. For example, a `WM_LBUTTONDOWN` message might be mapped to an `OnLButtonDown` handler function in a view object. When the message is received, the message map calls the handler function.

A `CWinApp` object is responsible for the message polling.

4.1 MESSAGES HANDLING IN MFC

Microsoft Windows works by sending messages to the windows that constitute an application’s user interface. The MFC library provides a unique way to map one or more of these messages to functions that you write. The first step in adding message mapping is to declare a message map in the class’s header file. Placing the line `DECLARE_MESSAGE_MAP()` at the end of the class’s declaration, just before the closing brace, takes care of this easy task. `DECLARE_MESSAGE_MAP()` is a macro defined by MFC. This macro takes care of all the details required to declare a message map for a class.

After you have declared your message map in your class’s header, it’s time to define the message map in your class’s implementation file. You start the message map with a `BEGIN_MESSAGE_MAP()` macro that takes two arguments; the name of the class for which the message map is being defined and the name of the base class. An `END_MESSAGE_MAP()` macro marks the end of the message loop.

```
BEGIN_MESSAGE_MAP(DerivedClassName, BaseClassName)
```

```
// Add message map entries here
```

```
END_MESSAGE_MAP()
```

No semicolon is required in `BEGIN_MESSAGE_MAP()` and `END_MESSAGE_MAP()`.

Now, you declare the message handling functions with an `afx_msg` prefix in your application main window class definition file. The `afx_msg` part of the prototype does nothing more than mark the function as a message map function.

To declare any message handling functions into your AppWizard application, better is to use the ClassWizard to write any message handling functions. The use of ClassWizard is shown in figure 4.1.

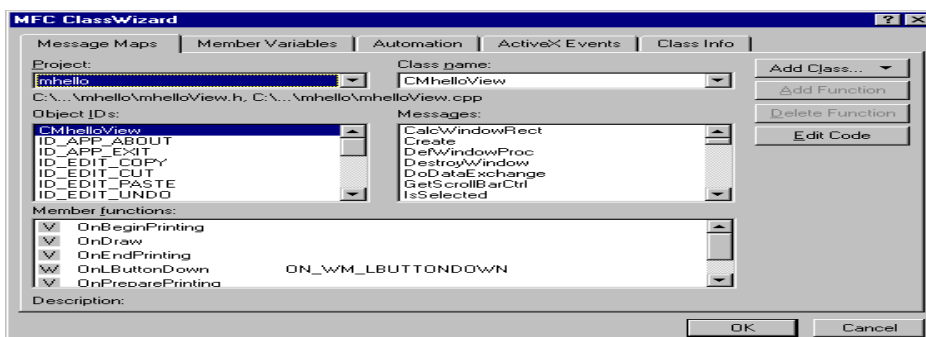


Figure 4.1 Define Message maps using ClassWizard

Go to the message map property sheet button, select your class name from the classes pool, select as desired your message ID number and click on the add function button or double-click on message type.

The listing of modify programs discussed in chapter 3 “Documents and Views” are written here with the message handling functions (WM_LBUTTONDOWN) .

Listing of MHELLOVIEW.H header file with message entries.

```
class CMhelloView : public CView
{
...
// Generated message map functions
protected:
   //{{AFX_MSG(CMhelloView)
    afx_msg void OnLButtonDown(UINT nFlags, CPoint point);
   //}}AFX_MSG
    DECLARE_MESSAGE_MAP()
};
```

Listing of MHELLOVIEW.CPP header file with message map.

```
...
BEGIN_MESSAGE_MAP(CMhelloView, CView)
   //{{AFX_MSG_MAP(CMhelloView)
    ON_WM_LBUTTONDOWN()
   //}}AFX_MSG_MAP
....
END_MESSAGE_MAP()
...
// CMhelloView message handlers
void CMhelloView::OnLButtonDown(UINT nFlags, CPoint point)
{
    AfxMessageBox("Welcome! you have pressed Left mouse button.");
    CView::OnLButtonDown(nFlags, point);
}
```

The output of the program is shown in figure 4.2.

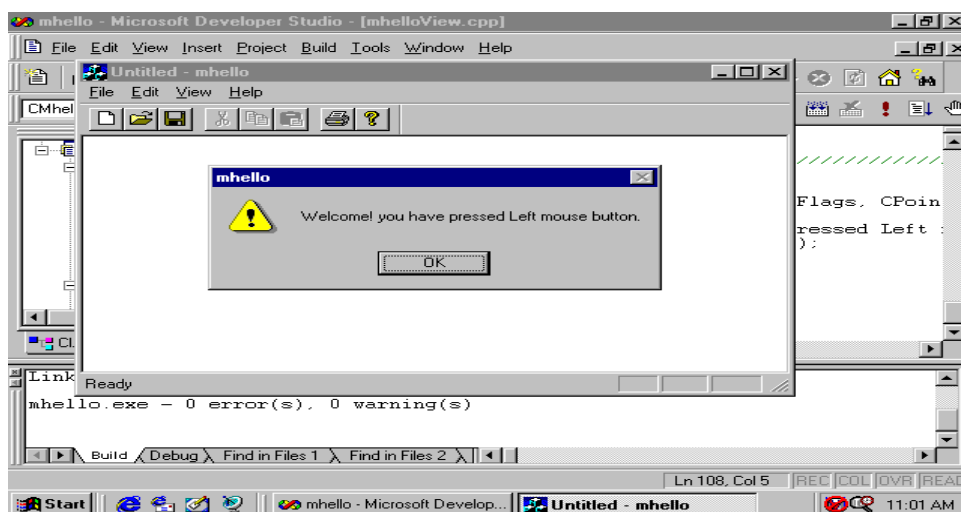


Figure 4.2 Application output

4.2 EXERCISE

1. Implements all client and non-client mouse messages.
2. Draw a free hand drawing in the window client area.
3. Write a MFC based application to demonstrate all the message box options.

5. MENU AND KEYBOARD ACCELERATORS

A Windows menu is familiar application element that consists of a top-level horizontal list of items with associated pop-up menus that appear when user selects a top-level item.

Menus allow you to arrange commands in a logical, easy-to-find fashion. With the Microsoft Developer Studio menu editor, you can create and edit menus by working directly with a menu bar that closely resembles the one in your finished application.

With the menu editor, you can:

- Create standard menus and commands.
- Assign shortcut keys, accelerator keys, and status bar prompts to menus and commands.

In addition, you can use ClassWizard to hook menu items to code.

The menu editor, with its various components labeled, is shown in Figure 5.1.

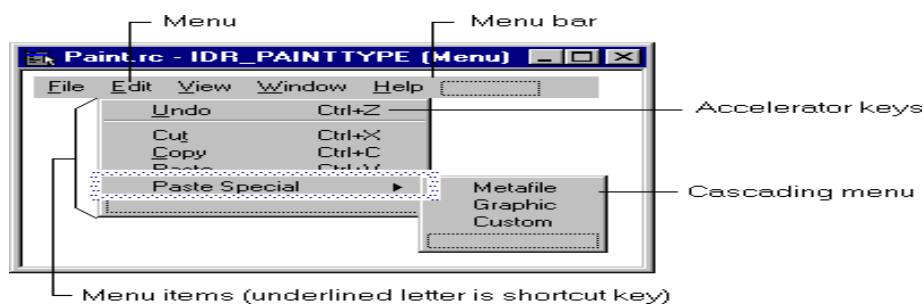


Figure 5.1 Menu Editor Components

5.1 STANDARD MENU AND COMMANDS

You can create menus, cascading menus, and menu commands on the menu bar in the menu editor.

5.1.1 To Create a Menu on the menu bar

- 1 Select the new-item box (an empty rectangle) on the menu bar (see Figure 5.2). Or move the new-item box to a blank spot with the RIGHT ARROW and LEFT ARROW keys.

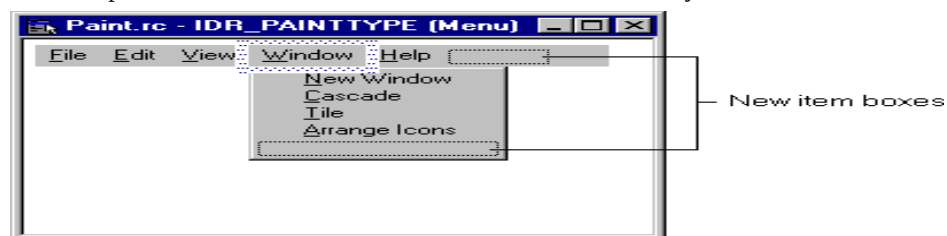


Figure 5.2 Menu Editor New-Item Boxes

- 2 Type the name of the menu. When you start typing, focus automatically shifts to the Menu Item property page, and the text you type appears both in the Caption box and in the menu editor window.

You can define a mnemonic key that allows the user to select the menu with the keyboard. Type an ampersand (&) in front of a letter to specify it as the mnemonic. Make sure all the mnemonics on a menu bar are unique.

Once you have given the menu a name on the menu bar, the new-item box shifts to the right, and another new-item box opens below for adding menu items.

5.1.2 To create a menu item

- 1 First, create a menu according to the steps outlined in the previous procedure.
- 2 Select the menu's new-item box.
–or–
Select an existing menu item and press the INS KEY. The new-item box is inserted before the selected item.
- 3 Type the name of the menu item. When you start typing, focus automatically shifts to the Menu Item property page, and the text you type appears in the Caption box.
You can define a mnemonic key that allows the user to select the menu command. Type an ampersand (&) in front a letter to specify it as the mnemonic. The user can select the menu command by typing that letter.
- 4 In the ID box, type the menu item ID, or select an existing command identifier. If you don't specify an ID, Visual C++ will generate an ID for you based on the command name.

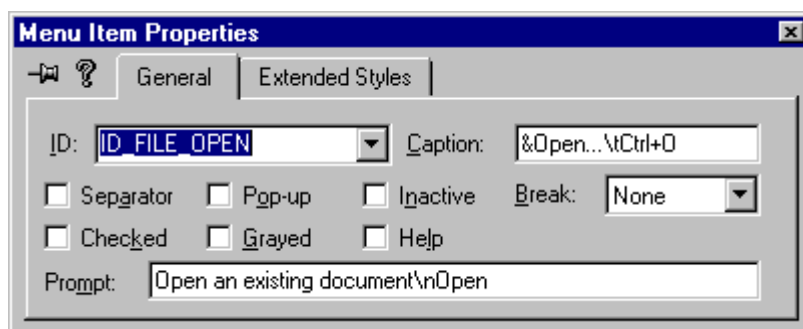


Figure 5.3 Menu Item Properties Window

- 5 On the property page, select the menu item styles that apply.
- 6 In the Prompt box on the property page, type the prompt string you want to appear in your application's status bar.
This feature is only available with Microsoft Foundation Class Library resource script (.RC) files.
This creates an entry in the string table with the same resource identifier as the menu item you created.
- 7 Press ENTER to complete the menu item.

5.2 KEYBOARD ACCELERATORS

The keyboard accelerators is the shortcut key to activate the menu-item action. The keyboard accelerators resource consists of a table of key combinations with associated command IDs. The Edit Copy menu-item(with command ID ID_EDIT_COPY), for example, might be linked to the Ctrl-C key combination through a keyboard accelerator entry.

5.2.1 Adding an Accelerator key with menu-item

Many times you want a menu item and a keyboard combination to issue the same program command. You do this by assigning the same resource identifier to the menu item and to an entry in your application's accelerator table. You then edit the menu item's caption to show the name of the accelerator key.

- 1 In the menu editor, select the menu item you want. From the Edit menu, choose Properties or double-click the item.
- 2 In the Caption box, add the name of the accelerator key to the menu caption:
 - Following the menu caption, type the escape sequence for a TAB (\t), so that all the menu's accelerator keys are left-aligned.
 - Type the name of the modifier key (CTRL, ALT, or SHIFT) followed by a plus sign (+) and the name, letter, or symbol of the additional key.

For example, to assign CTRL+O to the Open command on the File menu, you modify the menu item's caption so that it looks like this:

Open\tCtrl+O

- 3 Create the accelerator-table entry in the accelerator editor and assign it the same identifier as the menu item. Use a key combination that you think will be easy to remember.

5.3 EXERCISE

1. Define a menu structure and accelerators into an MFC based application.

<u>M</u> aster	<u>T</u> ransactions	<u>R</u> eports	<u>H</u> elp
<u>A</u> /c master	<u>B</u> ank Ctrl B	<u>L</u> edger Ctrl L	<u>A</u> bout FA Ctrl H
<u>G</u> roups	<u>C</u> ash Ctrl C	<u>T</u> rial Ctrl T	<u>S</u> etup Ctrl S
<u>S</u> ubgroup	<u>J</u> ournal Ctrl J	<u>P</u> /L Report	<u>Q</u> uit Ctrl X

6. GRPAHICS AND TEXT DRAWING

6.1 THE GRAPHICS DEVICE INTERFACE(GDI)

The set of functions for doing drawing is called the GDI(Graphics Device Interface), which is a DLL supplied with the operating system. If you want to draw any graphics or texts in the client area or to print a graphics or text on printer, it must use the GDI functions, which resides in the GDI32.DLL file. The GDI is a program file called GDI.EXE that is stored in the \WINDOWS\SYSTEM directory. The Windows environment will load GDI.EXE into memory when it is needed for graphics output.

6.2 THE DEVICE CONTEXT

The Windows operating system provides device independence through the device context concept. Windows OS programs do not send data directly to the screen or printer. Instead, the program obtains a handle for the screen or printer's device context.

The device context is a set of data maintained by Windows OS and identified by a device context handle. A device context not only represents the destination entity for the drawing operations, but also holds information about the drawing objects and parameters that specify the modes that the GDI drawing functions use when drawing to the device context.

A device context represents one of the following: the video display, the printer or plotter, or a memory device. MFC based applications generally use two methods for getting a DC handle in preparation for painting the screen : OnDraw() and OnPaint() method.

6.3 GDI OBJECTS

The device context contains handles to drawing objects. The drawing objects are : a brush, a pen, a font, a bitmap.

The MFC class CGdiObject provides a base class for the drawing objects. Your never create a CGdiObject object directly. Rather, you create an object from one of its derived classes, which are:

- CPen Create a pen.
- CBrush Create a brush object
- CFont Select or create a font.
- CBitmap Select or create a bitmap(BMP) file.
- CPalatte
- CRgn.

6.4 THE CDC CLASS

The **CDC** class defines a class of device-context objects. The **CDC** object provides member functions for working with a device context, such as a display or printer, as well as members for working with a display context associated with the client area of a window. This is the base class for all the device context classes.

To use a **CDC** object, construct it, and then call its member functions that parallel Windows functions that use device contexts.

6.5 THE DEVICE CONTEXT CLASSES

A device context is a Windows data structure that contains information about the drawing attributes of a device such as a display or a printer. All drawing calls are made through a device-context object, which encapsulates the Windows APIs for drawing lines, shapes, and text. Device contexts allow device-independent drawing in Windows. Device contexts can be used to draw to the screen, to the printer, or to a metafile.

MFC provides several device context classes derived from CDC, which are:

- CClientDC
- CPaintDC
- CWindowDC
- CMetaFileDC.

CPaintDC objects encapsulate the common idiom of Windows, calling the **BeginPaint** function, then drawing in the device context, then calling the **EndPaint** function. The **CPaintDC** constructor calls **BeginPaint** for you, and the destructor calls **EndPaint**. The simplified process is to create the **CDC** object, draw, and destroy the **CDC** object. In the framework, much of even this process is automated. In particular, your OnDraw function is passed a **CPaintDC** already prepared (via **OnPrepareDC**), and you simply draw into it. It is destroyed by the framework and the underlying device context is released to Windows upon return from the call to your OnDraw function.

CClientDC objects encapsulate working with a device context that represents only the client area of a window. The CClientDC constructor calls the **GetDC** function, and the destructor calls the **ReleaseDC** function.

CWindowDC objects encapsulate a device context that represents the whole window, including its frame.

CMetaFileDC objects encapsulate drawing into a Windows metafile. In contrast to the **CPaintDC** passed to OnDraw, you must in this case call **OnPrepareDC** yourself.

Here, I have written a program to display “Hello Windows” message into their client area. The complete source code of this application is printed below for your reference. You should be clear that to write under client area of any application, only the CDC device context to be used. By default, OnDraw() member function of your view class is used to display the desired message.

The Listing for a OnDraw() member function of a CmhelloView class.

```
void CMhelloView::OnDraw(CDC* pDC)
{
    CMhelloDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);
    pDC->TextOut(100,100,"Hello Windows");
}
```

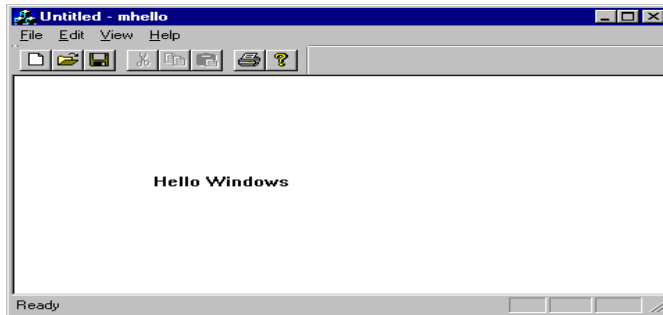


Figure 6.1 The output of the programme

Just try yourself, the same program using yours own defined message handling functions, for example any menu item or mouse handling messages.

6.6 CREATING A PEN

Pen drawing objects are used for drawing the outlines of the shapes drawn by the functions `Rectangle()`, `Ellipse()`, `Pie()`, `LineTo()`, and `Arc()`. Pens can specify lines of any pixel width. When a `CPen` object is constructed, the pen style, the pen width in pixels, and the color must be specified.

Pen Style	Description
PS_SOLID	Creates a Solid pen.
PS_DASH	Creates a dashed pen.
PS_DOT	Creates a dotted pen.
PS_DASHDOT	Creates a pen with alternating dashes and dots.
PS_DASHDOTDOT	Creates a pen with alternating dashes and double dots.
PS_NULL	Creates a Null pen.
PS_INSIDEFRAME	Creates a pen that draws a line inside the frame of closed shapes or inside lines.

Table 6.1 Pen styles with their description

6.7 GRAPHICS OBJECTS

Object Name	Description
<code>Arc()</code>	Draw position of an Ellipse.
<code>Ellipse()</code>	Draw an Ellipse.
<code>MoveTo()</code> , <code>LineTo()</code>	Draw a line from old coordinates to new coordinates.
<code>Pie()</code>	Draw a Pie.
<code>Rectangle()</code>	Draw a rectangle.
<code>Polyline()</code>	Draw a Polyline.

Table 6.2 Graphics Objects

6.8 EXCERCISE

1. Write an application to draw “Welcome users in Visual C++” in the client area of a window, where the background color is WHITE and text color as RED.
2. Write an application to draw all graphics objects one-by one into your client area.
3. Modify the `rectangle()` function to create a rectangle figure using the `MoveTo()` and `LineTo()` functions. Fill the rectangle with a green brush and draw the lines and the rectangle border with a blue pen.

Hints:

1. To create a Pen Object

```
CPen ppen(PS_SOLID,3,RGB(255,255,0));
```

Or

```
CPen ppen;
```

```
Ppen.Create(PS_SOLID,3,RGB(255,0,0));
```

2. To set a Text color in Red

```
CClientDC dc;
```

```
dc.SetTextColor(RGB(255,0,0));
```

```
dc.TextOut(100,100,"Red Color");
```

7. WINDOWS CONTROL

In a Windows application, dialog boxes are the most common object used to get information from the user to the program. Of course, dialog boxes aren't much good without the many controls that can be placed in them. Edit controls, list boxes, combo boxes, radio buttons, check boxes, and other types of controls all work together to provide the application's user with convenient ways to enter data into a program.

There are several types of controls you can place in a dialog box, including check boxes, radio buttons, list boxes, combo boxes, and scroll bars. You should already be familiar with how these controls work, both from a user's and a programmer's point of view. If you've never programmed with Microsoft Foundation Classes, you may not be familiar with the classes with which MFC encapsulates each of the window controls. These classes include CStatic, CButton, CEdit, CListBox and CComboBox.

A description of each window control follows:

- *Static text* is a string of characters usually used to label other controls in a dialog box or window. Although it is considered to be a window control, static text cannot be manipulated by the user. You use the CStatic class to create and manipulate static text.
- An *Edit control* accepts text input from the user. The user can edit the text in various ways before completing the input. You use the CEdit class to create and manipulate an edit box.
- A *Button* is a command object that triggers a command when the user clicks it. You use the CButton class to create and manipulate pushbuttons.
- A *List box* is a rectangle containing a set of selections. These selections are usually text items but can also be bitmaps or other objects. Depending on the list box's style flags, the user may select one or several objects in the list box. You use the CListBox class to create and manipulate a list box control.
- A *Combo box* is similar to a list box, except it also includes an edit control in which the user can type a selection. You use the CComboBox class to create and manipulate combo boxes.
- A *Scroll bar* is a graphical object containing a track that encloses a sliding box called the scroll box. By positioning the scroll box, the user can select a value from a given range. In addition to the scroll box, a scroll bar contains arrow boxes that, when clicked, move the scroll box a unit in the direction of the arrow. Although scroll bars are rarely used in dialog boxes, they can be created and manipulated by the CScrollBar class.

You have probably taken advantage of App Studio's tools for setting up buttons, scroll bars, edit controls, and other controls in dialog boxes. To derive or implement these controls into your application(client area), App Studio's can't help here. It's usually easier to create a control from one of the MFC control classes, and to position the control in your main window.

7.1 BUTTON CONTROL

Buttons are rectangular windows that usually contain a caption describing the use of the control. MFC provides all the functionality of the standard Windows button in the class CButton.

Buttons are among the most prevalent controls in Windows applications. When you think of a button, you may think of the OK or Cancel pushbutton in a typical dialog box, but there are four general styles of buttons defined in class CButton :

- Checkboxes
- Pushbuttons
- Radio buttons

- Group boxes
 - A *Check box* is a special type of button that toggles a check mark when clicked. Check boxes usually represent program options that the user can select.
 - A *Pushbutton* is a command object that triggers a command when the user clicks it.
 - *Radio buttons* are similar to check boxes, except only one radio button in a group can be selected at a given time. Radio buttons usually represent program options that are mutually exclusive.
 - *Group box* Often, check boxes and radio buttons are placed into group boxes, which organize the buttons into logical groups. The user cannot interact with group boxes.

MFC also provides class `CBitmapButton` for buttons that display bitmaps on their client areas. A `CBitmapButton` is derived from `CButton`, that means it inherits all the capabilities of a `CButton` object.

7.1.1 Button Styles

Like all windows, buttons can have a combination of window styles; because buttons are windows, they can use any combination of `CWnd` styles.

Style Macro	Meaning
BS_3STATE	This style is just like a checkbox, but the box can be checked or dimmed.
BS_AUTO3STATE	This style is just like a checkbox, but the box can be checked or dimmed. When a user selects the box, the check state of the button toggles automatically.
BS_AUTOCHECKBOX	This style is just like a checkbox, but when a user selects the box, the checked state of the button toggles automatically.
BS_AUTORADIOBUTTON	This style is the same as the radio button. When a user selects the circle, the button automatically selects itself and deselects any other radio buttons with the same style in the same group.
BS_CHECKBOX	This style creates a checkbox.
BS_DEFPUSHBUTTON	This style creates a default pushbutton that allow the user to quickly select the default command option by pressing the Enter key.
BS_GROUPBOX	This style creates a captioned framing rectangle for visually grouping controls.
BS_PUSHBUTTON	This style creates a pushbutton.
BS_RADIOBUTTON	This style creates a radio button. Buttons with this style are usually used in groups of related but mutually exclusive choices.

Table 7.1 Button Styles

7.1.2 Button Messages

MFC message map have to handle all the button notification messages. A button sends notification messages to its owner, which is usually a `CDialog` derived class. These messages can be trapped and handled by writing message map entries and message-handler methods for each message.

A message-map entry uses the following basic format :

```
ON_Message(ControlID,ClassMethod)
```

In this syntax, Message is the message to handle, ControlID is the child window ID of the button control sending the message, and ClassMethod is the name of the parent class method that will handle the message.

The method prototype in the owner's class declaration is as follow :

```
afx_msg void ClassMethod();
```

The message map entries useful for button controls are listed in Table 7.2.

Message Map Entry	Meaning
ON_BN_CLICKED	Sent by a button control when a user clicks the button.

Table 7.2 Button Message maps

7.1.3 Creating a Button

1. Allocate an instance of a CButton or CBitmapButton object by declare there defination into the view class.
2. Initialize the button object with its respective Create() method to attach a standard Windows button to it and set Button parameters and styles.

```
CButton* pButton = new CButton;
```

```
BOOL Create(DWORD bstyle,const RECT& rc,CWnd* pWnd,UINT nID);
```

Example : To Create and Implement a button object.

Figure 7.1 Shows the
button control



33

```

#endif // _MSC_VER >= 1000
#define ID_WELCOME 100 // Button ID
class CButtonView : public CView
{
protected: // create from serialization only
    CButtonView();
    DECLARE_DYNCREATE(CButtonView)
// Attributes
public:
    CButtonDoc* GetDocument();
    CButton wel;
// Operations
public:
// Overrides
    // ClassWizard generated virtual function overrides
    //{AFX_VIRTUAL(CButtonView)
    public:
        virtual void OnDraw(CDC* pDC); // overridden to draw this view
        virtual BOOL PreCreateWindow(CREATESTRUCT& cs);
    protected:
        //}AFX_VIRTUAL
// Implementation
public:
    virtual ~CButtonView();
#ifdef _DEBUG
    virtual void AssertValid() const;
    virtual void Dump(CDumpContext& dc) const;
#endif
protected:
// Generated message map functions
protected:
    //{AFX_MSG(CButtonView)
    afx_msg void OnButton();
    afx_msg void OnCombo();
    afx_msg void OnEdit();
    afx_msg void OnList();
    afx_msg void OnWelcome();
    //}AFX_MSG
    DECLARE_MESSAGE_MAP()
};
#ifdef _DEBUG // debug version in buttonView.cpp
inline CButtonDoc* CButtonView::GetDocument()
{ return (CButtonDoc*)m_pDocument; }
#endif
////////////////////////////////////
//{{AFX_INSERT_LOCATION}}
// Microsoft Developer Studio will insert additional declarations immediately before the previous line.
#endif // !defined(AFX_BUTTONVIEW_H_BB21A12E_CD65_11D2_BFD4_A18FA73A4152__INCLUDED_)
// buttonView.cpp : implementation of the CButtonView class
//
#include "stdafx.h"
#include "button.h"
#include "buttonDoc.h"
#include "buttonView.h"
#ifdef _DEBUG
#define new DEBUG_NEW

```

```

#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif
////////////////////////////////////
// CButtonView
IMPLEMENT_DYNCREATE(CButtonView, CView)
BEGIN_MESSAGE_MAP(CButtonView, CView)
   //{{AFX_MSG_MAP(CButtonView)
    ON_COMMAND(ID_WELCOME, OnWelcome)
    ON_COMMAND(IDM_BUTTON, OnButton)
    ON_COMMAND(IDM_COMBO, OnCombo)
    ON_COMMAND(IDM_EDIT, OnEdit)
    ON_COMMAND(IDM_LIST, OnList)
   //}}AFX_MSG_MAP
END_MESSAGE_MAP()
////////////////////////////////////
// CButtonView construction/destruction
CButtonView::CButtonView()
{
    // TODO: add construction code here

}
CButtonView::~CButtonView()
{
}

BOOL CButtonView::PreCreateWindow(CREATESTRUCT& cs)
{
    // TODO: Modify the Window class or styles here by modifying
    // the CREATESTRUCT cs
    return CView::PreCreateWindow(cs);
}

////////////////////////////////////
// CButtonView drawing

void CButtonView::OnDraw(CDC* pDC)
{
    CButtonDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);
    // TODO: add draw code for native data here
}
////////////////////////////////////
// CButtonView diagnostics
#ifdef _DEBUG
void CButtonView::AssertValid() const
{
    CView::AssertValid();
}
void CButtonView::Dump(CDumpContext& dc) const
{
    CView::Dump(dc);
}
CButtonDoc* CButtonView::GetDocument() // non-debug version is inline
{
    ASSERT(m_pDocument->IsKindOf(RUNTIME_CLASS(CButtonDoc)));

```

```

        return (CButtonDoc*)m_pDocument;
    }
#endif // _DEBUG
// CButtonView message handlers
void CButtonView::OnButton()
{
    CRect rect(0,0,100,50);
    wel.Create("&Welcome",WS_VISIBLE|WS_CHILD,rect,this,ID_WELCOME);
}

void CButtonView::OnCombo()
{
    // TODO: Add your command handler code here
}

void CButtonView::OnEdit()
{
    // TODO: Add your command handler code here
}

void CButtonView::OnList()
{
    // TODO: Add your command handler code here
}

void CButtonView::OnWelcome()
{
    AfxMessageBox("You clicked Welcome button.");
}

```

7.2 EDIT CONTROL

An edit control is a simple looking rectangle window that uses to get information from user's end at the runtime. Edit controls performs many useful functions that make working with text data much easier than you might think. Edit controls can maintain their own scroll bars to allow the user to scroll through the text, and can display text on more than one line. There are two general styles of edit controls :

- Single-line edit control
- Multiple-line edit control

A Single line edit control displays all text on a single line, regardless of hard-return characters. Multiple-line edit controls display text on multiple lines, separated by hard return characters.

MFC provides the services of a standard Windows edit control in class CEdit.

7.2.1 Edit Control Styles

Style Macro	Meaning
ES_AUTOHSCROLL	Automatically scrolls text to the right by 10 characters when a user types a character at the end of the line. When the user presses the Enter key, the control scrolls all text back to positions zero.
ES_AUTOVSCROLL	Automatically scrolls text up one page when a user presses Enter on the last line.

ES_CENTER	Centers text in a multiple-line edit control.
ES_LOWERCASE	Automatically converts all characters to lowercase as they are typed into the edit control.
ES_MULTILINE	Specifies that an edit control is a multiple-line edit control.
ES_UPPERCASE	Converts all characters to uppercase as they are typed into the edit control.
ES_READONLY	Prevents a user from entering or editing text in the edit control.

Table 7.3 Edit Control Styles

7.2.2 Edit Messages

Because MFC wraps the standard Windows edit messages into CEdit class method, an MFC program usually handles only notification messages. In an MFC program, you map messages that a CEdit object deals with to class methods. Table 7.3 shows the CEdit message notifications:

Message Map Entry	Meaning
ON_EN_CHANGE	Sent when a user has changed text in an edit control.
ON_EN_MAXTEXT	Sent when the number of characters in an edit control has exceeded the specified number of character for the edit control and text has been truncated..
ON_EN_KILLFOCUS	Sent when an edit control loses the input focus.

Table 7.4 Edit Message entries

7.2.3 CEdit Class Methods

The CEdit class offers a well-rounded set of methods for manipulating text in an edit window. The default mode for an edit control displays all edit text on a single line.

Method	Description
Clear()	Deletes the current selection from the edit control.
Cut()	Cuts the current selection from the edit control to the Clipboard in CF_TEXT format.
Paste()	Inserts the data from the Clipboard into the edit control at the current cursor position. Data is inserted only if the clipboard contains data in CF_TEXT format.
Undo()	Reverse the last edit-control operation.
Copy()	Copies the current selection from the edit control to the Clipboard in CF_TEXT format.

Table 7.5 Edit Control methods

7.2.4 Creating an Edit Control

To create a CEdit object, you use the two-step construction process typical of MFC.

1. Allocate an instance of a CEdit object by calling the constructor CEdit::CEdit() using the C++ keyword new.
2. Initialize the CEdit object and attach a Window edit control to it with the CEdit::Create() method to set edit control parameters and styles.

Try yourself to implement an edit control for single line as well as multiline edit control based on the previous button control.

Example :

```
#define IDC_EDITSING      128
#define IDC_EDITMULTI    129
CEdit* pSing = new CEdit;
CEdit* pMulti = new CEdit;
pSing->Create(ES_SINGLE|WS_VISIBLE|WS_CHILD|ES_LEFT | WS_BORDER, CRect(200,10,500,30)
, this, IDC_EDITSING);
```

```
pMulti->Create(ES_MULTILINE|WS_VSCROLL|WS_VISIBLE|WS_CHILD|ES_LEFT |WS_BORDER,
CRect(200,10,500,30), this, IDC_EDITMULT);
```

7.3 LIST BOX CONTROL

A list box is a rectangular window that usually contains a list of strings. List boxes are very common user interface controls that allow the selection of one or more items from a list. List boxes maintain their own scroll bars to allow the user to scroll among the data in the list.

MFC provides all of the functionality of a standard Windows list box in the class `CListBox`. The `CListBox` class provides a set of methods that make working with list box items relatively easy. A list box can be created as a child control of any window by writing code but is typically defined in a dialog resource template.

7.3.1 List Box Styles

Style Macro	Meaning
LBS_NOTIFY	The list box sends its parent an input message when a user clicks or double-clicks a list item.
LBS_SORT	The list box alphabetically sorts strings in the list.
LBS_STANDARD	This style is a combination of four style bits : LBS_NOTIFY LBS_SORT WS_VSCROLL WS_BORDER. The list box draws a border around itself, sorts strings in the list alphabetically, and sends its owner an input message when a user clicks or double-clicks a list item.

Table 7.6 List Box Styles

7.3.2 ListBox Messages

Because MFC wraps the standard Windows list box messages such as `LB_INSERTSTRING` or `LB_ADDSTRING` into `CListBox` class methods, an MFC program usually needs to handle only notification messages. A list box with the `LBS_NOTIFY` style sends notification messages to its owner, which is usually a `CDialog` derived class.

Message Map Entry	Meaning
ON_LBN_DBLCLK	List boxes with the <code>LBS_NOTIFY</code> style send this message to the owner when a user double clicks an item in a list box.
ON_LBN_SELCHANGE	List boxes with the <code>LBS_NOTIFY</code> style send this message to the owner when the current list box selection has been canceled.
ON_LBN_KILLFOCUS	This message occurs when a list box loses the input focus.

Table 7.7 List box Message entries

7.3.3 CListBox Class Methods

The `CListBox` class provides many methods for dealing with and manipulating list boxes and list box data.

Method	Description
<code>GetCount()</code>	Gets the number of items in a list box.
<code>GetText()</code>	Copies a list box string into a buffer.
<code>AddString()</code>	Adds a string to a list box.
<code>DeleteString()</code>	Deletes a string from a list box.
<code>InsertString()</code>	Inserts a string into a specific index in a list box.

Table 7.8 List box methods

7.3.4 Creating a List Box

To create a List Box, Here I am giving the full listings of the programs `LISTBOXVIEW.H` and `LISTBOXVIEW.CPP` for your references. In this example, I have implemented a listbox contains the names of all CEDTs, and double click on any list item, it gives you the selected CEDT name.

```

// listboxView.h : interface of the CListboxView class
///////////////////////////////////////////////////////////////////
#ifdef !defined(AFX_LISTBOXVIEW_H__B19B474E_D153_11D2_BFD4_DD74A3D4F564__INCLUDED_)
#define AFX_LISTBOXVIEW_H__B19B474E_D153_11D2_BFD4_DD74A3D4F564__INCLUDED_
#if _MSC_VER >= 1000
#pragma once
#endif // _MSC_VER >= 1000
#define ID_LISTBOX 128
class CListboxView : public CView
{
protected: // create from serialization only
    CListboxView();
    DECLARE_DYNCREATE(CListboxView)
// Attributes
public:
    CListboxDoc* GetDocument();
    CListBox listbox;
    // ClassWizard generated virtual function overrides
   //{{AFX_VIRTUAL(CListboxView)
public:
    virtual void OnDraw(CDC* pDC); // overridden to draw this view
    virtual BOOL PreCreateWindow(CREATESTRUCT& cs);
protected:
    //}}AFX_VIRTUAL

// Implementation
public:
    virtual ~CListboxView();
#ifdef _DEBUG
    virtual void AssertValid() const;
    virtual void Dump(CDumpContext& dc) const;
#endif
// Generated message map functions
protected:
    //{{AFX_MSG(CListboxView)
    afx_msg void OnListbox();
    afx_msg void OnListSelected();
    //}}AFX_MSG
    DECLARE_MESSAGE_MAP()
};
#ifdef _DEBUG // debug version in listboxView.cpp
inline CListboxDoc* CListboxView::GetDocument()
{ return (CListboxDoc*)m_pDocument; }
#endif
//{{AFX_INSERT_LOCATION}}
// Microsoft Developer Studio will insert additional declarations immediately before the previous line.
#endif // !defined(AFX_LISTBOXVIEW_H__B19B474E_D153_11D2_BFD4_DD74A3D4F564__INCLUDED_)

// listboxView.cpp : implementation of the CListboxView class
//
#include "stdafx.h"
#include "listbox.h"
#include "listboxDoc.h"
#include "listboxView.h"
#ifdef _DEBUG
#define new DEBUG_NEW

```

```

#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif
////////////////////////////////////
// CListBoxView
IMPLEMENT_DYNCREATE(CListBoxView, CView)
BEGIN_MESSAGE_MAP(CListBoxView, CView)
   //{{AFX_MSG_MAP(CListBoxView)
    ON_COMMAND(IDC_LISTBOX, OnListbox)
    ON_LBN_DBLCLK(ID_LISTBOX, OnListSelected)
   //}}AFX_MSG_MAP
END_MESSAGE_MAP()
CListBoxView::CListBoxView()
{
    // TODO: add construction code here
}

CListBoxView::~CListBoxView() { }

BOOL CListBoxView::PreCreateWindow(CREATESTRUCT& cs)
{
    // TODO: Modify the Window class or styles here by modifying
    // the CREATESTRUCT cs
    return CView::PreCreateWindow(cs);
}

void CListBoxView::OnDraw(CDC* pDC)
{
    CListBoxDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);
}

#ifdef _DEBUG
void CListBoxView::AssertValid() const
{
    CView::AssertValid();
}

void CListBoxView::Dump(CDumpContext& dc) const
{
    CView::Dump(dc);
}

CListBoxDoc* CListBoxView::GetDocument() // non-debug version is inline
{
    ASSERT(m_pDocument->IsKindOf(RUNTIME_CLASS(CListBoxDoc)));
    return (CListBoxDoc*)m_pDocument;
}
#endif // _DEBUG
void CListBoxView::OnListbox()
{
    CRect rc(5,5,100,100);
    listbox.Create(WS_CHILD | WS_VISIBLE | LBS_NOTIFY, rc, this, ID_LISTBOX);
    listbox.AddString(CString("Mohali"));
    listbox.AddString(CString("Gorakhpur"));
    listbox.AddString(CString("Jammu"));
}

```



```

listbox.AddString(CString("Calicut"));
listbox.AddString(CString("Gouhati"));
listbox.AddString(CString("Aurangabad"));
rc = CRect(4,4,101,101);
CClientDC dc(this);
dc.Rectangle(rc);
}
void CListView::OnListSelected()
{
int nSel= listbox.GetCurSel();
CString mtxt;
listbox.GetText(nSel,mtxt);
AfxMessageBox(mtxt);
}

```

The output of this programme is show in figures 7.2 and 7.3.



Figure 7.2 The main window contains a List box into window client area with all CEDT names.

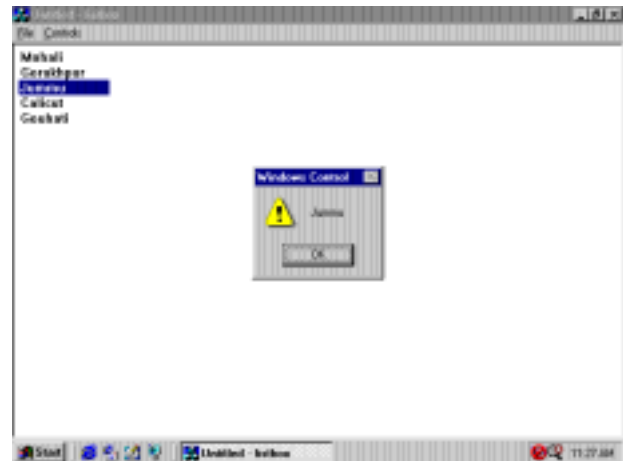


Figure 7.3 The main window displays a message box with the highlighted string from the listbox

7.4 COMBO BOX CONTROL

A combo box gives you all the power of a list box combined with an edit control, by rolling them up into a single control. A combo box can use the static text control instead of an edit box to prevent a user from editing selected text. The list box part of the control can either dropdown in response to a user selecting the drop-down arrow at the right side of the control, or can remain visible at all times.

If a list box contains items, and an item is currently selected in the list box portion of the control, then the selected item is displayed in the combo box's companion control. There are three general styles available for a combo box :

- Simple combo
- Drop-down combo
- Drop-down list

A simple combo always has its list box displayed and uses an edit control as the companion control. A drop-down combo displays its list when dropped down and uses an edit control as the companion control. A drop-down list displays its list when dropped down and uses a static text control as its companion control.

MFC provides all of the functionality of a standard Window combo box in the class CComboBox. The CComboBox class provides a set of methods that make working with list box items relatively easy. A combo box can be created as a child control by using code, but is typically defined in a dialog resource template.

7.4.1 Combo Box Styles

Style Macro	Meaning
CBS_AUTOHSCROLL	Automatically scrolls the text in the combo box's child edit control to the right when a user types a character at the end of the line. If this style is not set, only text that fits within the rectangular boundary is allowed.
CBS_DROPDOWN	The list box portion is not displayed unless a user selects the drop down button next to the edit control; the current selection in the list box portion is displayed in the edit control .
CBS_SORT	Automatically sorts list box strings in a combo box.
CBS_SIMPLE	The list box portion is always visible and the current selection in the list box is displayed in the combo box child edit control.
CBS_DROPDOWNLIST	The list box portion is not displayed unless a user selects the drop-down button next to a static text item, which displays the current selection in the list box.

Table 7.9 Combo Box styles

7.4.2 Combo Box Messages

Because MFC wraps the standard Windows combo box messages such as CB_FINDSTRING or CB_SELECTSTRING into CComboBox class methods, an MFC program usually needs to handle only notification messages. A combo box sends notification messages to its owner, which is usually a CDialog-derived class.

Message Map Entry	Meaning
ON_CBN_DBLCLK	Sent when the user double-clicks a list item in a combo box with the CBS_SIMPLE style..
ON_CBN_DROPDOWN	Sent when the list box of a combo box is about to drop-down, exposing the list items.
ON_CBN_SETFOCUS	The combo box receives the input focus.
ON_CBN_SELENDOK	Sent when a user selects an item and then close the list. This notification message is sent before the CBN_CLOSEUP message to indicate that the user's selection should be considered valid.
ON_CBN_KILLFOCUS	This message occurs when a combo box loses the input focus.

Table 7.9 Combo box Message entries

8.2.1 7.4.3 CComboBox Class Methods

The CComboBox class provides a full set of methods for manipulating lists in a combo box.

Method	Description
Clear()	Deletes any selected text from the edit control.
Cut()	Cuts any selected text from the edit control and copies it to the Clipboard in CF_TEXT format.
Paste()	Inserts the data from the Clipboard into the edit control portion of the combo box.
GetCount()	Gets the number of items in the list box of a combo box.
LimitText()	Limits the length of the text that a user can enter into the edit control portion of a combo box.
AddString()	Adds a string to the end of the list in the list box portion of a combo box.
DeleteString()	Deletes a string from a list box portion of a combo box.
InsertString()	Inserts a string into the list box of a combo box.
Copy()	Copies the current selection from the edit control to the Clipboard in CF_TEXT format.

Table 7.10 Combo box methods

7.4.4 Creating a Combo Box

To create a CComboBox object, you use the two-step construction process typical of MFC.

1. Allocate an instance of a CComboBox object by calling the constructor CComboBox::CComboBox() using the C++ keyword new.
2. Initialize the CComboBox object with a call to the CComboBox::Create() method.

```
CComboBox* pCombo = new CComboBox;  
pCombo->Create(CBS_DROPDOWN|WS_VSCROLL|WS_CHILD|WS_VISIBLE,  
CRect(100,10,600,50),this, IDC_COMBOBOX);
```

Try yourself to implement a combo box with the help of above listbox program.

7.5 EXERCISE

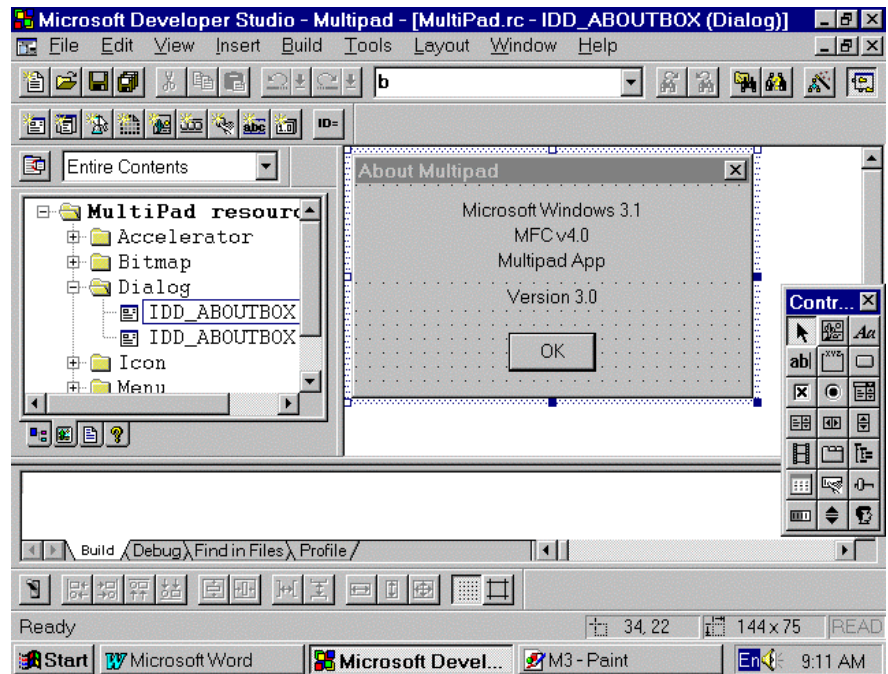
1. Study and implements the Edit control as single line and multi-line edit control.
2. Study and implement the Static control and Scroll bars.
3. Write a program to make a editor as a multi-line using edit control in both forms(SDI and MDI).
4. Write a program to create an application to add a string, delete a string and find a string using List box and edit control.
5. Draw a password screen for your project.
6. Write a program to implements a List box control properties and events into an application.
7. Write a program to implements a Combo box control properties and events into an application.
8. Implement Edit Control as a Single line and Multi-line and sets their maximum characters up to 200 characters only. Also converts each character into uppercase automatically.
9. Write a MFC based application, to enter the personal information of the enroll student's into your institute(Enter Student Name, Sex, Date of Joining, Course Name and Fee etc).

8. DIALOG BOXES

8.1 INTRODUCTION TO DIALOG BOX

A dialog box usually contains a set of dialog controls, themselves child windows, through which the user and the application exchange data.

Figure 8.1 A dialog box shown in Microsoft Developer Studio



To facilitate the efficient use of dialog boxes, Windows provides two types:-

- Modal dialog boxes
- Modeless dialog boxes

8.1.1 Modal Dialog

When an application displays a modal dialog box, the window that owns the dialog box is disabled, effectively suspending the application. The user must complete interaction with the modal dialog before the application can continue.

8.1.2 Modeless Dialog

In contrast to modal dialogs, presenting a modeless dialog does not suspend execution of the application by disabling the owner window of the dialog box. However, modeless dialog remain on top of their owner windows even when the owner window gains focus.

8.1.3 Dialog Templates

Dialog templates are typically created as part of the application's resource file. They can be created manually as set of instructions in the resource file, or they can be created through a visual resource file editor, such as the resource editor of the Developer Studio.

The MFC Library supports dialogs through the **CDialog** class and derived classes. A CDialog object corresponds to a dialog window, the content of which is based on a dialog template resource.

8.2 CONSTRUCTING DIALOGS

The basic steps in constructing a dialog and making it part of your application include creating the dialog template resource, creating a CDialog-derived class that corresponds to this resource, and constructing an object of this class at the appropriate location in your application. To create a dialog class corresponding to the dialog shown in your current window viewport, use the right mouse button anywhere in the dialog editor window to bring up a pop-up menu; from this popup menu, select the Class Wizard command or from *View | ClassWizard*.

The ClassWizard, after detecting that it has been invoked for a newly constructed resource, presents the adding a class dialog, select the Create a new class radio button and click OK.

At this time, the Class Wizard displays the New Class dialog. Here, you can enter the dialog's name and set other options, such as filename, the resource identifier, or others settings. Add a suitable name for the new class, for example, CNewDialog.

```
class CNewDialog : public CDialog
{
public :
    CNewDilaog(CWnd* pParent=NULL);
    enum {IDD=ID_NEWDIALOG};
protected :
    virtual void DoDataExchange(CDataExchange* pDx);
    DECLARE_MESSAGE_MAP();
};
CNewDialog::CNewDialog(CWnd* pParent) : CDialog(CNewDialog::IDD,pParent)
{ }
CNewDialog::DoDataExchange(CdataExchange* pDx)
{CDialog::DoDataExchange(pDX); }
```

8.3 INVOKING THE DIALOG

8.3.1 Modal Dialog Box

To invoke a created dialog class in your application view window. First, to invoke through a menu-item selection. Secondly, through a dialog box child control selection.

We invoke the dialog via the DoModal function.

```
void CMainFrame::OnModalDialog()
{
    CNewDialog mdlg;                // defined a newly, created dialog box class
    if (mdlg.DoModal()==IDOK)        // Activate the dialog box
        AfxMessageBox("You have pressed OK button in a Modal Dialog box");
}
```

8.3.2 Modeless Dialog Box

To invoke a modeless dialog box, we must first modify the dialog's constructor function. In the constructor, we must make a call to the Create member function in order to construct the dialog box object.

```
CNewDialog::CNewDialog(CWnd* pParent)
```

```

{
    Create(CNewDialog::IDD,pParent);
}

```

Invocation, of the dialog from CMainFrame::OnViewDialog is also different. Instead of calling the dialog's DoModal member function, we just construct the dialog object; the call to Create within the constructor takes care of the rest.

```

void CMainFrame::OnViewDialog()
{
    CNewDialog* pNewDlg;
    pNewDlg = new CNewDialog;
    pNewDlg->ShowWindow(SW_SHOW);
}

```

8.3.3 Dialog Data Exchange

Dialog Data Exchange takes place in the dialog class's DoDataExchange member function. In this function, calls are made for all member variables that are mapped to controls. The class that are made are to a family of MFC functions with their names that begins with DDX_.

For example, to perform data exchange between an edit control and a member variable of type CString, the following call is made:

```
DDX_Text(pDX, IDC_NAME,m_Name);
```

8.3.4 Dialog Data Validation

In addition to the simple exchange of data between member variables and dialog controls, MFC also offers a data validation mechanism. Data validation is accomplished through calls to functions with names that begins with DDV_.

An example for a data validation function is DDV_MaxChars, which is used to validate the length of a string typed into an edit control. To validate that a string in an edit control is no longer than 50 characters, you would make the following call:

```
DDV_MaxChars(pDX,m_Name,50);
```

8.4 COMMON DIALOGS

Common dialogs have been a feature of Microsoft Windows. They are used for opening and closing files, selecting colors, fonts, performing text searches, and setting up and using the printer.

MFC supports for common dialogs is provided through classes derived from the class **CCommonDialog**, which is itself derived from CDialog.

8.4.1 CColorDialog

The CColorDialog class supports the Windows color selection dialog. To display a color dialog, construct a CColorDialog object and call its DoModal function.

```

void CMainFrame::OnColorDialog()
{
    CColorDialog dlg;
    if (dlg.DoModal()==IDOK)
    {

```



Figure 8.2 Color Window

```

TCHAR temp[80];
wsprintf(temp,_T("Color Selected : %8.8X",dlg.GetColor()));
AfxMessageBox(temp);
}
}

```

The figure 8.2 shows the Color Dialog box output.

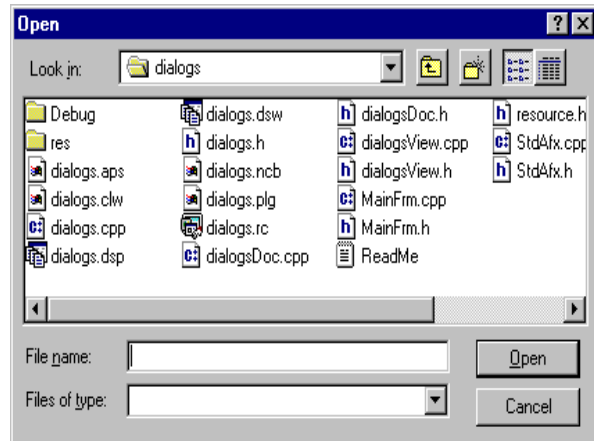
8.4.2 CFileDialog

The CFileDialog class supports the Windows Open and Save As dialog boxes.

```

void CMainFrame::OnFileOpenDialog()
{
    CFileDialog dlg(TRUE);
    if (dlg.DoModal()==IDOK)
    {
        .....
        .....
        .....
    }
}

```



The figure 8.3 shows the Open a file dialog box.

Figure 8.3 Open a file

8.4.3 CFindReplaceDialog

The CFindReplaceDialog class supports the use of the windows Find and Replace dialog in MFC application.

Use of the Find and Replace dialog is fundamentally different from the user of other common dialogs. This dialog box is a modeless dialog.

```

void CMainFrame::OnFindReplaceDialog()
{
    CFindReplaceDialog *pdlg = new CFindReplaceDialog;
    pdlg->Create(TRUE,_T("Find Dialog"),NULL,FR_DOWN,this);
}

```

The figure 8.4 shows Find and Replace dialog box output.

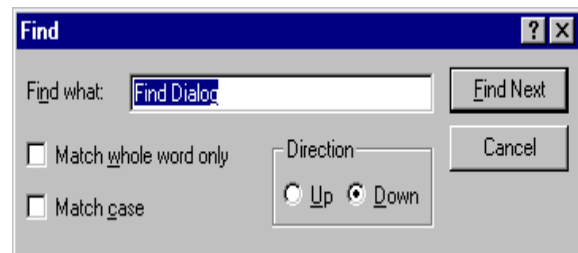


Figure 8.4 Find and Replace window

8.4.4 CFontDialog

The CFontDialog class supports font selection in MFC applications through the windows font selection common dialog.

To use the font dialog, create a CFontDialog object and call its DoModal member function.

```

void CMainFrame::OnFontDialog()
{
    CFontDialog dlg;

```

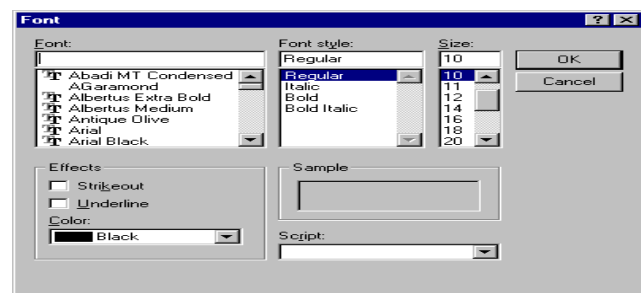


Figure 8.4 Font Dialog box

```

if (dlg.DoModal()==IDOK)
{
    CString temp = _T("Font Selected : ");
    temp=temp+dlg.GetFaceName();
    AfxMessageBox(temp);
}
}

```

The figure 8.4 shows the output of a Font window.

8.4.5 CPageSetupDialog

The Page Setup dialog is used under Windows 95 and Windows NT 3.51 to setup the printed page. This dialog replaces the Print Setup dialog.

To use the Page Setup dialog, create an object of type CPageSetupDialog and call its DoModal member function.

```

void CMainFrame::OnPageSetupDialog()
{
    CPageSetupDialog dlg;
    if (dlg.DoModal()==IDOK)
    {
        .....
        .....
    }
}

```

The Figure 8.5 shows the Page Setup dialog box.

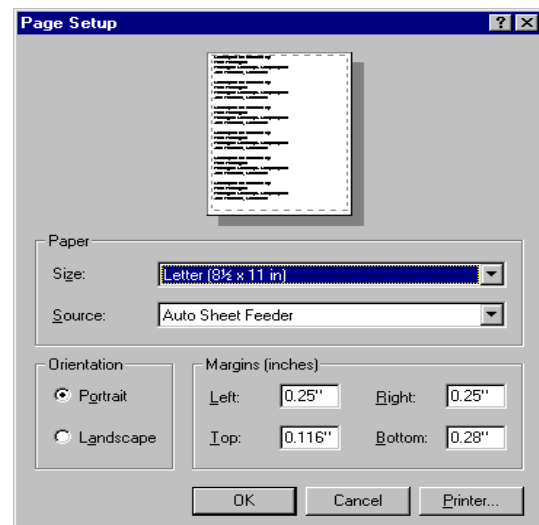


Figure 8.5 Page Setup

8.4.6 CPrintDialog

The CPrintDialog class supports the use of the Print and Print Setup dialogs in MFC applications. To create a Print dialog, construct a CPrintDialog object and call its DoModal member function.

```

void CMainFrame::OnPrintDialog()
{
    CPrintDialog dlg(FALSE);
    if (dlg.DoModal()==IDOK)
    {
        // Printing commands
    }
}

```

The figure 8.6 shows the Print dialog box.

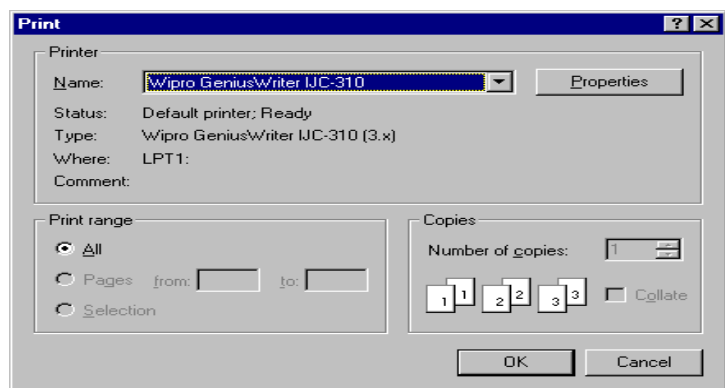
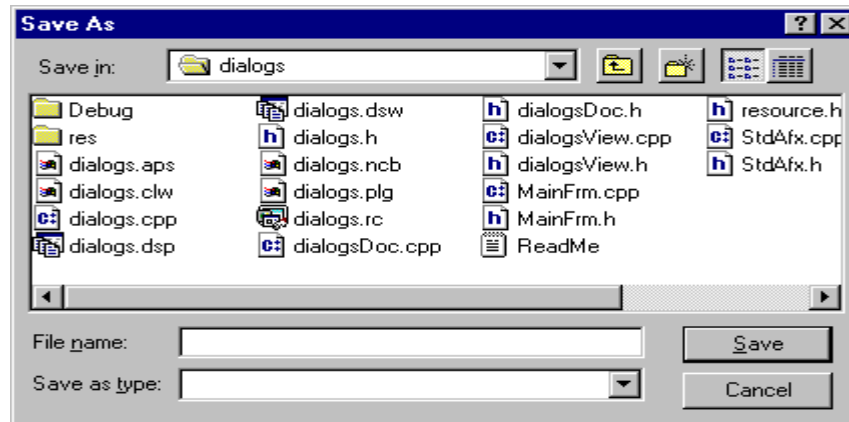


Figure 8.6 Print dialog box

8.5 EXERCISE

1. Define and implement a Modal dialog box.
2. Define and implement a Modeless dialog box.
3. Define and implements all custom-control dialog boxes- Open a File, Save As, Color, Font, Printer Setup and Print.
4. Define a File open and File save dialog boxes where the filters are : Text files, RTF files and All files only.
5. Implements a Save As dialog box, and check that the filename should not be exist into the current folder(shown in below figure).



9. FILE ACCESS

MFC provides the CFile class and its derived classes, CStdioFile class, CMemFile and CSharedFile, for the file input/output for disk-based and memory based files.

9.1 INTRODUCTION TO CFile CLASS

MFC provides support for files and file operation through the CFile class, which is the base class for all MFC file classes. Classes derived from CFile provide support for text file.

```
class CFile : public CObject
{
public :
    CFile();
    CFile(LPCTSTR lpszFilename,UINT nOpenFlags);
    virtual ~CFile();
    UINT m_hFile;
    virtual CString GetFileName() const;
    virtual CString GetFilePath() const;
    virtual CString GetFileTitle() const;
    virtual BOOL Open(LPCTSTR lpszFileName,UINT nOpenFlags, CFileException* pError=NULL);
    ....
    virtual void Read(void *lpBuff, UINT nCount);
    virtual void Write(const void *lpBuff, UINT nCount);
    virtual void Close();
};
```

9.1.1 CFile::CFile

```
CFile cf;
CFile cf( int hFile );
CFile cf( LPCTSTR lpszFileName, UINT nOpenFlags );
```

Parameters

hFile	The handle of a file that is already open.
lpszFileName	A string that is the path to the desired file. The path can be relative or absolute.
nOpenFlags	Sharing and access mode. Specifies the action to take when opening the file. You can combine options listed below by using the bitwise-OR () operator. One access permission and one share option are required; the modeCreate and modeNoInherit modes are optional. The values are as follows: · CFile::modeCreate Directs the constructor to create a new file. If the file exists already, it is truncated to 0 length. · CFile::modeNoTruncate Combine this value with modeCreate. If the file being created already exists, it is not truncated to 0 length. Thus the file is guaranteed to open, either as a newly created file or as an existing file. This might be useful, for example, when opening a settings file that may or may not exist already. This option applies to CStdioFile as well. · CFile::modeRead Opens the file for reading only. · CFile::modeReadWrite Opens the file for reading and writing. · CFile::modeWrite Opens the file for writing only.

- CFile::modeNoInherit Prevents the file from being inherited by child processes.
- CFile::typeText Sets text mode with special processing for carriage return–linefeed pairs .
- CFile::typeBinary Sets binary mode.

Example 1

```
char* pFileName = "sonitest.dat";
TRY
{
    CFile f( pFileName, CFile::modeCreate | CFile::modeWrite );
}
CATCH( CFileException, e )
{
    CString mstr= T("File could not be opened ") + e->mcause;
    AfxMessageBox(mstr);
}
END_CATCH
```

9.1.2 CFile::Open

virtual **BOOL** Open(LPCTSTR lpszFileName, UINT nOpenFlags, CFileException* pError = NULL);

Return Value *Nonzero if the open was successful; otherwise 0. The pError parameter is meaningful only if 0 is returned.*

Parameters

- lpszFileName A string that is the path to the desired file. The path can be relative or absolute but cannot contain a network name.
- nOpenFlags A UINT that defines the file's sharing and access mode. It specifies the action to take when opening the file. You can combine options by using the bitwise-OR (|) operator. One access permission and one share option are required; the modeCreate and modeNoInherit modes are optional. See the CFile constructor for a list of mode options.
- pError A pointer to an existing file-exception object that will receive the status of a failed operation.

Example 2

```
CFile f;
CFileException e;
char* pFileName = "sonitest.dat";
if( !f.Open( pFileName, CFile::modeCreate | CFile::modeWrite, &e ) )
{
    CString mstr= T("File could not be opened ") + e->mcause;
    AfxMessageBox(mstr);
}
```

9.1.3 CFile::Read

virtual **UINT** Read(void* lpBuf, **UINT** nCount);

Return Value The number of bytes transferred to the buffer. Note that for all CFile classes, the return value may be less than nCount if the end of file was reached.

Parameters

- lpBuf Pointer to the user-supplied buffer that is to receive the data read from the file.
- nCount The maximum number of bytes to be read from the file. For text-mode files, carriage return–linefeed pairs are counted as single characters.

Example 3

```
CFile cfile;
```

```
char pbuf[100];
UINT nBytesRead = cfile.Read( pbuf, 100 );
```

9.1.4 CFile::Write

```
virtual void Write( const void* lpBuf, UINT nCount );
```

Parameters

LpBuf	A pointer to the user-supplied buffer that contains the data to be written to the file.
nCount	The number of bytes to be transferred from the buffer. For text-mode files, carriage return–linefeed pairs are counted as single characters.

Example 4

```
CFile cfile;
char pbuf[100];
cfile.Write( pbuf, 100 );
```

9.2 THE CStdioFile CLASS

MFC provides the CStdioFile class to represents a buffered C runtime stream files as opened by the C runtime function fopen(). This file can be opened in either text mode or in binary mode.

The CSdtioFile class provides only two additional methods for text I/O operations:

```
* ReadString()  Reads a single line of text.
* WriteString() Writes a single line of text.
```

Example : To Implement ReadString() member function

```
CStdioFile fr;
CStdioFile fw;
CFileException fe;
CString stemp;
char* lpszFilename = "SONI.DAT";
if(!fr.Open(LPCTSTR) lpszFilename,Cfile::modeRead | Cfile::typeText,&fe))
{
    TRACE("Can't open file %s error = %u\n",&lpszFilename,fe.m_cause);
}
fr.ReadString(stemp);
```

9.3 THE CMemFile CLASS

The CMemFile class represents a file that resides completely in RAM, and therefore has no corresponding disk file. Memory file act just like disk files except that the file is stored in RAM instead on disk.

9.4 EXERCISE

1. Create and open a text file. Also reads and writes text into this file.
2. Creates and opens a binary file. Also reads and writes a raw data into this file.
3. Implements all file operations- Copy file, Rename a file, Moving a file and Delete a file.
4. Write a small database package using File Input\Output functions.
 - Add a New record
 - Delete a Record

- List of the Records
 - Search a Record
5. Develop an MFC based application to draw a open a file dialog box to accept source file and destination file names and to run the Windows Explorer functions like: Copy a file, Moving a file, Saving a file and Deleting a file.

10. DYNAMIC LINK LIBRARIES

Dynamic Link Library (DLL) is a binary file that provides a library of functions, objects and resources that can be shared or used by multiple processes simultaneously. There are several reasons to create and use DLL in your application's including these:-

- Sharing common code between different executable files.
- Breaking an application into its component parts to provide a way to easily upgrade application components separately.
- Keeping resource data out of an application executable, but still readily accessible to the application. An example of this is the creation of a separate DLL for each of each of the various languages an application supports. Each DLL could contain sting tables localized for a specific languages.

10.1 THE DLL ENTRY POINT

DllMain() is the entry point for every Dynamic Link Libraries. Win32 processes can be multithread. DllMain() is called every time a thread attaches database. Calls to DllMain() are made of three four cases:

- Process Attach: A process is attaching to the DLL.
- Process Detach: A process is detaching from the DLL
- Thread Attach: A thread is attaching to the DLL
- Thread Detach: A thread is detaching from the DLL

Example:

```
BOOL APIENTRY DllMain (HANDLE hInstance , ULONG reason, LPVOID reserved)
{
    switch(reason) {
        case DLL_PROCESS_ATTACH:
            //-----
            break;
        case DLL_PROCESS_DEATTACH:
            //-----
            break;
        case DLL_THREAD_ATTACH:
            //-----
            break;
        case DLL_THREAD_DEATTACH:
            //-----
            break;
    }
    return TRUE ;
}
```

10.2 DLLS WRITTEN WITH MFC

There are two types of DLLs MFC supports:

10.2.1 Regular DLLs

A regular DLL can link either statically or dynamically with MFC. Regular DLLs use the CWinApp class to wrap the Win32 API DllMain() function and setup an MFC shell for the DLL.

10.2.2 Extension DLLs

An extension DLL can link dynamically only to an MFC application. Just write AFX_EXT_CLASS in front of your function.

```
class AFX_EXT_CLASS CMyClass : public COBJ
{
public:
//...
};
```

10.3 HOW TO CREATE A DLL

To create a DLL using MFC, select a new project and select "**MFC AppWizard(*dll)**" from the project type. When you have selected the "Create" then you will select the options as per your DLL's creation from the next shown dialog box on your screen:

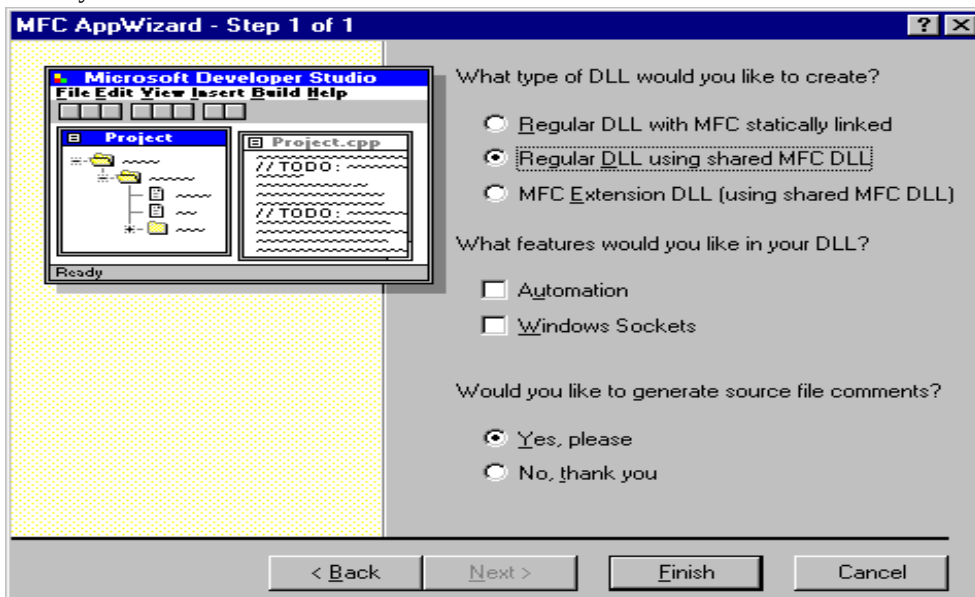


Figure 10.1 MFC DLLs Options

Select your option and press Finish button. It will create the basic files for your application. To create a simple DLL, I have selected a regular DLL with MFC statically linked and for your review, the code for the WELCOME.DLL file – WELCOME.H and WELCOME.CPP appears in Listing 10.1, WELCOME.DEF appears in Listing 10.2 and WELCOME.CPP in Listing 10.3.

Listing 10.1 welcome.h : main header file for the WELCOME DLL

```
#if !defined(AFX_WELCOME_H_530ADF07_D2E9_11D2_BFD4_E2AE6F31C8DB__INCLUDED_)
#define AFX_WELCOME_H_530ADF07_D2E9_11D2_BFD4_E2AE6F31C8DB__INCLUDED_
#if _MSC_VER >= 1000
#pragma once
#endif // _MSC_VER >= 1000
```

```

#ifndef __AFXWIN_H__
    #error include 'stdafx.h' before including this file for PCH
#endif
#include "resource.h"           // main symbols
extern "C" void FAR PASCAL _export Print(CView FAR* pView);
// CWelcomeApp
// See welcome.cpp for the implementation of this class
class CWelcomeApp : public CWinApp
{
public:
    CWelcomeApp();
// Overrides
    // ClassWizard generated virtual function overrides
   //{{AFX_VIRTUAL(CWelcomeApp)
   //}}AFX_VIRTUAL

   //{{AFX_MSG(CWelcomeApp)
        // NOTE - the ClassWizard will add and remove member functions here.
        // DO NOT EDIT what you see in these blocks of generated code !
   //}}AFX_MSG
    DECLARE_MESSAGE_MAP()
};
////////////////////////////////////
//{{AFX_INSERT_LOCATION}}
// Microsoft Developer Studio will insert additional declarations immediately before the previous line.
#endif // !defined(AFX_WELCOME_H__530ADF07_D2E9_11D2_BFD4_E2AE6F31C8DB__INCLUDED_)

```

Listing 10.2 welcome.cpp : Defines the initialization routines for the DLL.

```

#include "stdafx.h"
#include "welcome.h"
#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif
BEGIN_MESSAGE_MAP(CWelcomeApp, CWinApp)
   //{{AFX_MSG_MAP(CWelcomeApp)
        // NOTE - the ClassWizard will add and remove mapping macros here.
        // DO NOT EDIT what you see in these blocks of generated code!
   //}}AFX_MSG_MAP
END_MESSAGE_MAP()
extern "C" void FAR PASCAL _export Print(CView FAR* pView)
{
    CClientDC dc(pView);
    CString str_out("Welcome to my first Dlls using MFC");
    dc.TextOut(10,10,str_out);
}
////////////////////////////////////
// CWelcomeApp construction
CWelcomeApp::CWelcomeApp()
{
    // TODO: add construction code here,
    // Place all significant initialization in InitInstance
}
////////////////////////////////////
// The one and only CWelcomeApp object

```



```
CWelcomeApp theApp;
```

Listing 10.3 welcome.def : Declares the module parameters for the DLL.

```
LIBRARY      'welcome'  
DESCRIPTION  'welcome Windows Dynamic Link Library'  
CODE         PRELOAD MOVEABLE DISCARDABLE  
DATA        PRELOAD MOVEABLE SINGLE  
HEAPSIZE     1024  
EXPORTS      Print
```

10.4 EXERCISE

1. Create a Regular and Extension DLLs using MFC classes.
2. Also call these DLLs functions into your application.

11. DATABASE PROGRAMMING

Most business applications contains access and use databases. Most recently, however, information systems are being implemented using the client-server architecture, wherein the data is managed centrally by a server, and users access data stored in many different databases using the Structured Query Language (SQL). Open Database Connectivity System (ODBC) includes several components, one of which is the SQL-based Application Programming Interface (API).

11.1 ODBC OVERVIEW

Figure 11.1 illustrates the ODBC architecture that enables you to write a database application capable of accessing any database for which an ODBC driver is available. ODBC makes the application independent of the specific details of the database by providing an API that presents a table-orientated view of the underlying data, regardless of how the data is actually stored in the database.

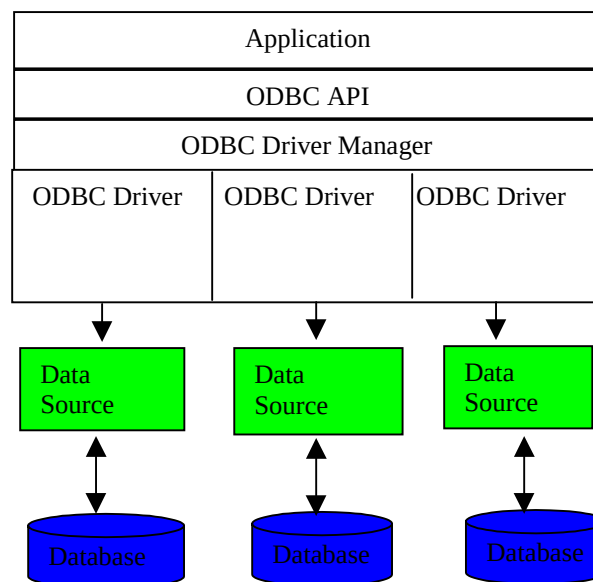


Figure 11.1 ODBC Architecture

The ODBC architecture consists of the following major components :-

- ODBC applications store and retrieve data by calling the ODBC vAPI functions. Usually, applications process and display the data in various formats that make the data more useful to the user.
- ODBC API provides a standard set of functions that applications can call to access data. The access method is similar to that provided by SQL.
- ODBC driver manager loads drivers on behalf of the application.
- ODBC driver is a DLL that processes ODBC function calls, send SQL statements to a specific database, and return results to the applications.
- Data Sources represents the files or database tables accessed by a driver.

11.2 MFC DATABASE CLASSES

MFC database classes are meant to simplifying writing database application with ODBC. There are six classes for database programming of which the following three are the most important :-

- CDatabase represents a connection to a data source.
- CRecordset models the tables in which results are returned by the ODBC driver.
- CRecordView is frames-based view of individual rows of the table represented by CRecordset.

These classes are well supported by the MFC AppWizard and ClassWizard, so much so that for most applications you do not even have to use these classes explicitly- the wizard do all the work for you.

11.3 CREATING DATABASE APPLICATION

When you use MFC AppWizard to generate your database application; you can see how the MFC classes simplify the problem of accessing and retrieving data from a ODBC data source.

Before you use AppWizard to generate an application with database support, ensure the following :-

- ODBC manager and drivers are installed.
- One ODBC data source is a setup with the ODBC manager (Control Panel), double click on the 32bits ODBC icon selected an ODBC drivers, and then pick a data source from the dialog box displayed by the screen.

Each steps is shown in the below figures, How to generate a ODBC application using the AppWizard.

1. In Step 2 of 6 as shown in the figure 11.2, select the Database view without file support and then click on the Data source button .

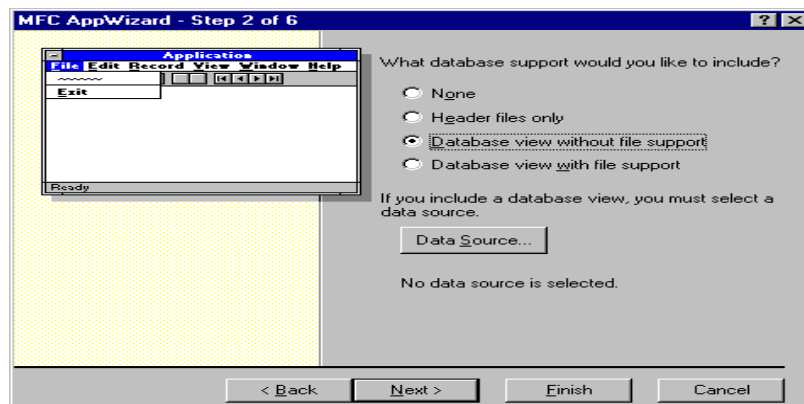


Figure 11.2
Database options

2. After click on the Data Source button, select the Database Option from the shown figure 11.3.

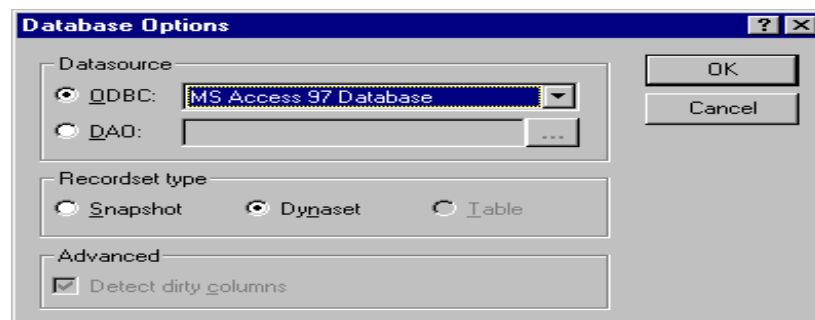


Figure 11.3
Database
Connection Type

3. Select the database from the Select database dialog shown in figure 11.4.

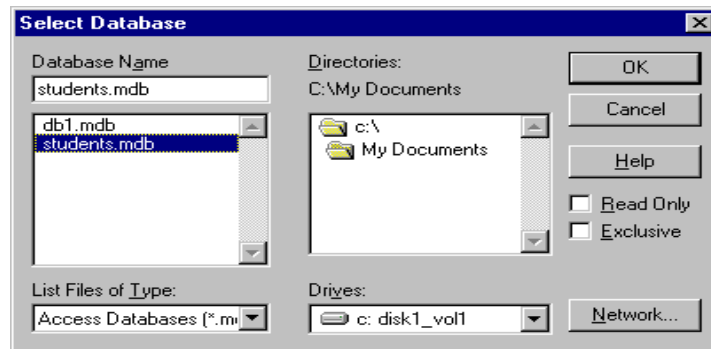


Figure 11.4 Select a Database

4. Select the Table name from the available tables under your selected database in the step 3.

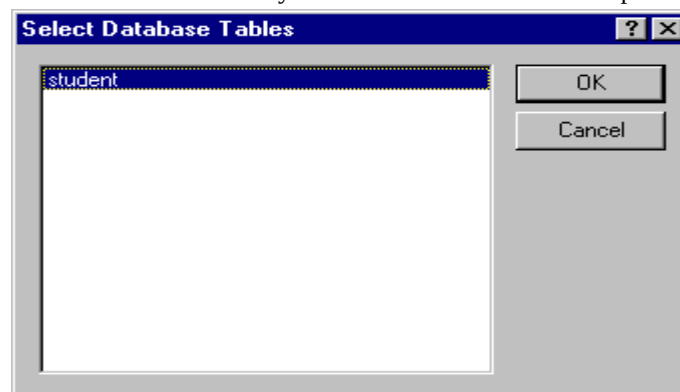


Figure 11.5 Select a Table

5. Now, design the empty dialog box for your database controls using the same steps to create a dialog box controls. Here, I have paste some controls for your selected Student Table.

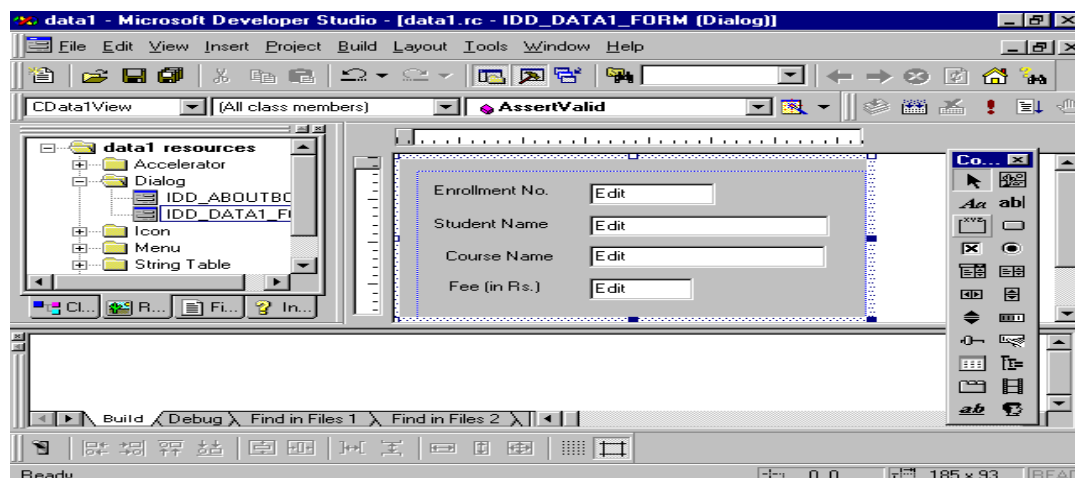


Figure 11.6 Design your Database Dialog box

6. Bound your dialog controls with your database fields, press CTRL+ENTER after select your control and define the desired database field as shown in figure 11.7

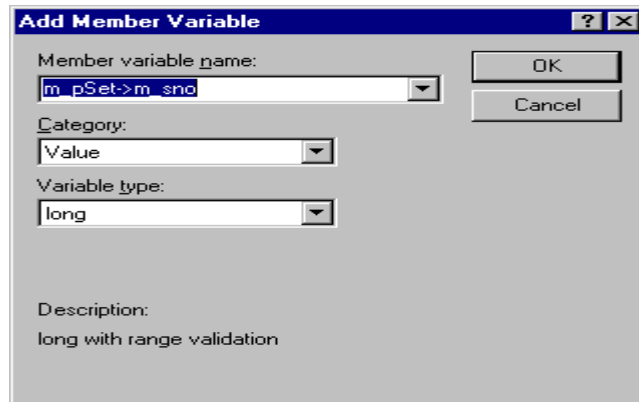


Figure 11.7 Add a Member Variable

7. Finally, compile your program and Execute it The Figure 11.8 shows the out window of your application.

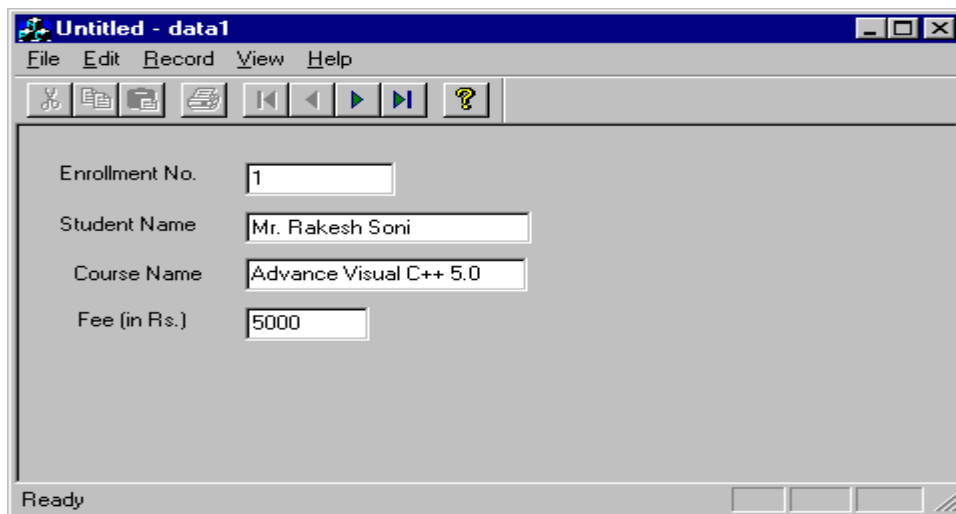


Figure 11.8 Output of your database application

11.4 DAO BASICS

You can use DAO to access data from local as well as external data sources. Using the MFC DAO classes will substantially reduce the complexity of your application. There are similar ODBC database classes- CDAODatabase, CDAORecordset, CDAORecordView.

Many databases, both local to a machine and external(remote databases), can be accessed by the use of either ODBC classes or DAO classes.

